

RH134

Improving Command-line Productivity



Red Hat | Enterprise Linux

Goal

- Run commands more efficiently by using advanced features of the Bash shell, shell scripts, and various utilities provided by Red Hat Enterprise Linux.

Objectives

- Automate sequences of commands by writing a simple shell script.
- Efficiently run commands over lists of items in a script or from the command-line using for loops and conditionals.
- Find text matching a pattern in log files and command output using the **grep** command and regular expressions.

What is the script?

- A script is nothing more than a list of system commands stored in an executable file. At the very least, this saves the effort of retyping that particular sequence of commands each time it is invoked.

مفهوم السكريبت (What is the script?)

1. التعريف الأساسي (Definition)

- **المعنى:** السكريبت (script) ليس أكثر من مجرد قائمة من أوامر النظام (system commands) المخزنة داخل ملف قابل للتنفيذ (executable file).
- **التفاصيل الفنية:** يتم حفظ الأوامر بترتيب معين داخل الملف لتعمل كحزمة واحدة متكاملة عند تشغيلها.

2. الفائدة والأهمية (Value & Benefit)

- **المعنى:** توفير الوقت والجهد في تنفيذ الأوامر المتكررة.
- **الأهمية:** يوفر على الأقل جهد إعادة كتابة (retyping) نفس تسلسل الأوامر في كل مرة تحتاج إلى تشغيلها (invoked)، مما يرفع من كفاءة الإنتاجية على سطر الأوامر.

What is the script?

- A script is nothing more than a list of system commands stored in an executable file. At the very least, this saves the effort of retyping that particular sequence of commands each time it is invoked.

Why shell scripting?

- A working knowledge of shell scripting is essential to anyone wishing to become reasonably proficient at system administration.
- Consider that as a Linux machine boots up, it executes the shell scripts in `/etc/rc.d` to restore the system configuration and set up services.
- The craft of scripting is not hard to master, since the scripts can be built in bite-sized sections and there is only a fairly small set of shell-specific operators and options to learn
- The syntax is simple and straightforward, similar to that of invoking and chaining together utilities at the command line.

1. **Essential Skill:** امتلاك معرفة عملية بالـ `Shell Scripting` يعد أمراً أساسياً لأي شخص يصبح كفاء في الـ `System Administration`.
2. **System Booting:** عند إقلاع جهاز الـ `Linux` يتم تنفيذ الـ `Shell Scripts` الموجودة في المسار `etc/rc.d/` لاستعادة الـ `System Configuration` وتجهيز الـ `Set Up Services`.
3. **Bite-Sized Sections:** كتابة الـ `Scripts` ليست صعبة؛ إذ يمكنك البدء بـ `Script` بسيط يجمع بعض أوامر النظام (مثل إنشاء مستخدمين أو فحص السيرفر) ثم تكبيره وتطويره خطوة بخطوة.
4. **Shell-Specific Operators:** الـ `Shell` تحتوي على مجموعة صغيرة ومحددة من الـ `Operators` والـ `Options` وليست معقدة أو ضخمة كباقي لغات البرمجة.
5. **Invoking and Chaining Together Utilities at the Command Line:** هي نفس الأوامر العادية التي تكتبها سطر بسطر على الشاشة، نحن فقط نجتمعها ونربطها معاً في ملف واحد لتنفيذ المهام بكفاءة.

When not to use shell scripts?

- Resource-intensive tasks, especially where speed is a factor (sorting, hashing, recursion ...).
- Procedures involving heavy-duty math operations, especially floating-point arithmetic, arbitrary precision calculations, or complex numbers (use C++ or FORTRAN instead).
- Need native support for multi-dimensional arrays.
- Need to generate/ manipulate graphics or GUIs.
- Need direct access to system hardware.
- Need to use libraries or interface with legacy code.

If any of the above applies, consider a more powerful scripting language -- perhaps Perl, Python, Ruby or possibly a compiled language such as C, C++, or Java.

1. Resource-Intensive Tasks (المهام كثيفة الاستهلاك للموارد):

- **التوضيح:** تجنب استخدام الـ `Shell Scripts` في المهام التي بتستهلك مجهود كبير من الـ `CPU` أو الـ `Memory`، خصوصاً لما تكون السرعة عامل أساسي (زي ترتيب بيانات ضخمة `sorting` أو تشفير `hashing` أو تكرار متداخل `recursion`).
- **ليه؟** لأن الشل في الآخر هو عبارة عن مفسر (`interpreter`) للأوامر بينفذ سطر بسطر ببطء نسبياً، ومش مخصص للعمليات الرياضية أو البرمجية الضخمة اللي محتاجة أداء صاروخي وسرعة فائقة زي اللغات المترجمة (`compiled languages`).

2. Heavy-Duty Math Operations (العمليات الحسابية المعقدة):

- **التوضيح:** تجنب العمليات الحسابية المعقدة زي الكسور العشرية الدقيقة (`floating-point arithmetic`)، أو الأعداد المركبة.
- **البديل:** لو مشروعك فيه حسابات هندسية أو علمية دقيقة جداً، استخدم لغات مخصصة للرياضيات وسريعة جداً زي `C++` أو `FORTAN`.

3. Multi-Dimensional Arrays (المصفوفات متعددة الأبعاد):

- **التوضيح:** لو الكود بتاعك محتاج يتعامل مع جداول بيانات معقدة أو مصفوفات بأبعاد متعددة (`Multi-dimensional arrays` زي الجداول المكونة من صفوف وأعمدة متداخلة).
- **إيه البديل؟** الـ `Bash` مبيتتعاملش مع المصفوفات دي بشكل أصلي ومرن (`Native support`)، وعشان تعملها فيه هتحتاج تعقيد برمجي رهيب، فالأفضل استخدام لغة ثانية زي `Python` أو `C++`.

4. Graphics and GUIs (الرسومات والواجهات):

- التوضيح: لو محتاجة تعملي برامج فيها واجهات مستخدم رسومية (GUIs) بأزرار وقوائم، أو ترسمي رسومات (Graphics).
- ليه؟ لأن الـ Shell Scripts أساسها شاشة سوداء أوامر نصية (CLI) فقط، وليست مصممة نهائياً لتصميم الواجهات أو معالجة الرسومات.

5. System Hardware (الوصول المباشر لعتاد النظام):

- المقصود بيها وأمثلة: لو محتاجة برمجة تتعامل مباشرة مع هاردوير الجهاز من غير وسيط (زي إنك تكتبي كود يخاطب كارت الشاشة GPU مباشرة، أو تقرئي بيانات الحساسات Sensors في المذربورد، أو تتحكمي في أذرع تحكم ألعاب Joysticks ، أو تتعاملين مع كارت ميموري خارجي على مستوى الـ Low-level Drivers).
- ليه؟ الشغل بيكلم النظام بشكل عام، لكنه مش مصمم للتحكم الدقيق والمباشر في قطع الهاردوير بالشكل ده؛ وهنا بيحتاجوا لغات قريبة من الهاردوير زي C أو ++C .

6. Legacy Code and Libraries (التفاعل مع الكود القديم والمكتبات):

- معنى "كود قديم" (Legacy Code): مش معناه كود مرمي في الزبالة! معناه نظام برمجي ضخم وقديم الشركات شغالين عليه من سنين ومبني بلغات قديمة (زي أنظمة البنوك أو السيرفرات القديمة المكتوبة بلغات زي C أو Fortran أو مكتبات برمجية جاهزة Libraries).
- إيه المشكلة؟ لو عندك نظام قديم كده، مينفعش تروح تربطه بـ Shell Script ضعيف؛ لأن الشغل مش هيعرف يقرأ المكتبات دي أو يتواصل مع الكود القديم ده بسلاسة، فيحتاج لغات برمجة قوية تدعم التوافقية دي.

7. Alternative Languages (اللغات البديلة):

- التوضيح: باختصار، لو مشروعك فيه أي حاجة من الحاجات السابقة (سرعة خارقة، حسابات معقدة، هاردوير، واجهات، أو كود قديم)، ارمي الـ Shell Script على جنب واعتمد على لغة برمجة أقوى بكتير زي Perl ، Python ، Ruby ، أو لغات مترجمة سريعة جداً زي C ، ++C ، أو Java .

Useful tips for writing effective bash scripts in Linux

- Always use comments in scripts.
- Make a script exit when fails.
- Make a script exit when bash uses undeclared variable.
- Use double quotes to reference variables.
- Use functions in scripts.
- Always perform debugging for long scripts.
- Make your functions and variable names more descriptive.



1. **Undeclared Variable**: إذا نسيت تعريف المتغير أو كتبت اسمه خطأ، سيتعامل معه الشل كقيمة فارغة (`null`) ويكمل التنفيذ، مما قد يتسبب في كوارث وحذف ملفات حساسة بالخطأ دون تحذير.
2. **Always Perform Debugging**: يجب تتبع واختبار السكريبتات الطويلة وفحصها جيداً قبل اعتمادها لتجنب الأخطاء المنطقية أو أخطاء الـ `syntax`.
 - **أمر التتبع**: `bash -x script.sh` (يُستخدم لعرض وتتبع كل أمر خطوة بخطوة أثناء تنفيذه لاكتشاف الأماكن التي يحدث فيها الخطأ بدقة).

What is a sha-bang?

- The sha-bang (`#!/`) at the head of a script tells your system that this file is a set of commands to be fed to the command interpreter indicated. سيتم توجيهها إلى مفسر الأوامر المحدد.
- Immediately following the sha-bang is a path name. This is the path to the program that interprets the commands in the script.
- This command interpreter then executes the commands in the script, starting at the top (line following the sha-bang line), ignoring comments.
- Note that the path given at the "sha-bang" must be correct, otherwise an error message -- usually "**Command not found.**" -- will be the only result of running the script.
- The default shell is `bash` in a Linux system – try "`echo $SHELL`" to display your current shell.

What is a sha-bang?

- This command interpreter then executes the commands in the script, starting at the top (line following the sha-bang line), ignoring comments.
- Note that the path given at the "sha-bang" must be correct, otherwise an error message -- usually "**Command not found.**" -- will be the only result of running the script.
- The default shell is **bash** in a Linux system – try "**echo \$SHELL**" to display your current shell.
- **#!** can be omitted if the script consists only of a set of generic system commands, using no internal shell directives.

• أوامر النظام العامة والأساسية التي اللينكس بيعرفها وبيفهمها بشكل افتراضي من غير ما يحتاج يروح لبرنامج معين يفسر هاله؛ لأن النظام كده كده عارف مكان الأوامر دي وبيفذها على طول.

يعني السكريبت بتاعك مبيستخدمش أي ميزات أو قواعد خاصة أو متقدمة تابعة للـ Shell نفسه (زي شروط الـ if/else ، أو حلقات التكرار for/while loops ، أو تعريف المتغيرات واستخدامها بطرق معقدة داخل الشل)

```
#!/bin/sh
#!/bin/bash
#!/usr/bin/csh
#!/usr/bin/perl
#!/usr/bin/python
```

يعني إيه؟ يعني لو السكريبت بتاعك عبارة عن كم أمر عادي جداً من بتوع اللينكس اللي بتكتبهم في العادي (زي ls أو cp أو mkdir أو pwd) ، فمش شرط تحطي سطر الـ sha-bang في الأول خالص .

Invoking the script

مشكلة `sh < scriptname` والـ `stdin:` استخدام علامة التوجيه < يعطل المدخلات القياسية ويقفل قناة التفاعل مع المستخدم، مما يجعل أي أمر داخل السكريبت ينتظر إدخالاً من الكيبورد مثل أمر `read` يتوقف ولا يعمل .

- Having written the script, you can invoke it by "**sh scriptname**" or alternatively "**bash scriptname**".
- But not recommended is using `sh <scriptname`, since this effectively disables reading from `stdin` within the script.
- Much more convenient is to make the script itself directly executable with a **chmod**.

`chmod 555 scriptname` (gives everyone read/execute permission).

`chmod +rx scriptname` (gives everyone read/execute permission).

`chmod u+rx scriptname` (gives only the script owner read/execute permission).

- Having made the script executable, you may now test it by **./scriptname**.
- If it begins with a "sha-bang" line, invoking the script calls the correct command • عند استخدام `bash scriptname` ، فأنت تتادين مفسر الباش ليقرأ الملف وينفذه سطر بسطر، interpreter to run it. وهذا لا يحتاج صلاحية تنفيذ (**execute**) بل يكفي صلاحية القراءة فقط. • أما عند التشغيل المباشر (`./scriptname`) ، فيتم معاملة السكريبت كبرنامج أو أمر مستقل، وهنا يُشترط تماماً وجود صلاحية التنفيذ. (**execute**)

Invoking the script

`chmod 555 scriptname` (gives everyone read/execute permission).

`chmod +rx scriptname` (gives everyone read/execute permission).

`chmod u+rx scriptname` (gives only the script owner read/execute permission).

- Having made the script executable, you may now test it by `./scriptname`.
- If it begins with a "sha-bang" line, invoking the script calls the correct command interpreter to run it.
- As a final step, after testing and debugging, you would likely want to move it to `/usr/local/bin` (as root, of course), to make the script available to yourself and all other users as a systemwide executable. The script could then be invoked by simply typing `scriptname` [ENTER] from the command line.

- الخطوة النهائية: بعد اختبار السكريبت والتأكد من عمله تماماً، يتم نقله إلى مسار النظام العام (`usr/local/bin/`) باستخدام صلاحيات الروت (`sudo`).

- الهدف: ليكون السكريبت متاحاً كأمر أساسي في النظام (`systemwide executable`) لجميع المستخدمين.

- طريقة الاستخدام: بعد النقل، يمكنك تشغيل السكريبت مباشرة بكتابة اسمه فقط من أي مكان في التيرمينال (مثل أوامر `ls` أو `cp`) دون الحاجة لكتابة `./` أو مسار الملف.

Quoting special characters

- A number of characters and words have special meaning to the Bash shell. However, occasionally you will want to use these characters for their literal values, rather than for their special meanings. To do this, use one of three tools to remove (or escape) the special meaning: **the backslash** (\), **single quotes** ('), or **double quotes** (").
- The backslash escape character removes the special meaning of the single character immediately following it. For example, to display the literal string **# not a comment** with the **echo** command, the # sign must not be interpreted by Bash as having special meaning. Place the backslash character in front of the # sign.

```
[user@host ~]$ echo # not a comment
```

```
[user@host ~]$ echo \# not a comment  
# not a comment
```


- When you need to escape more than one character in a text string, either use the escape character multiple times or employ single quotes ('). Single quotes preserve the literal meaning of all characters they enclose. Observe the escape character and single quotes in action:

```
[user@host ~]$ echo # not a comment #
```

```
[user@host ~]$ echo \# not a comment #  
# not a comment
```

```
[user@host ~]$ echo \# not a comment \  
# not a comment #
```

```
[user@host ~]$ echo '# not a comment #'  
# not a comment #
```

- "dot", as a component of a filename. When working with filenames, a leading dot is the prefix of a "hidden" file, a file that a ls will not normally show.

```
[root@server ~]# touch .hidden-file
```

- When considering directory names, a single dot represents the current working directory, and two dots denote the parent directory.

```
[root@server ~]# pwd  
/home/karim/projects  
[root@server ~]# cd ..  
[root@server ~]# pwd  
/home/karim
```


- The dot often appears as the destination (directory) of a file movement command, in this context meaning current directory.

```
[root@server ~]# cp /home/karim/projects/* .
```

- "dot" character match. When matching characters, as part of a regular expression, a "dot" matches a single character.
- ``command`` command substitution. The ``command`` construct makes available the output of command for assignment to a variable. This is also known as backquotes or backticks.

```
[root@Lab01 ~]# DATE=`date` == DATE=$(date)
[root@Lab01 ~]# echo $DATE
Tue Dec 22 21:35:27 EET 2020
```

- استبدال الأوامر: **(Command Substitution)** لتخزين نتيجة تنفيذ أمر داخل متغير (مش مجرد كتابة اسمه كنص)، لا يمكن استخدام علامات التنصيص المعتادة. ("")
- البديل الصحيح: يجب استخدام صيغة الأقواس \$ (أمر) أو علامات التراجع `امر`

- **!** It also inverts the meaning of a test operator. This can, for example, change the sense of equal (=) to not-equal (!=). The ! operator is a Bash keyword.
- ***** wild card [asterisk]. The * character serves as a "wild card" for filename expansion in globbing. By itself, it matches every filename in a given directory
The * also represents any number (or zero) characters in a regular expression.
- In the context of arithmetic operations, the * denotes multiplication.
- **?** wild card. ^{تعمل كـ} The ? character serves as a single-character "wild card" for filename expansion in globbing, as well as representing one character in an extended regular expression.

• يعني إيه **Wildcard** ؟ معناها "جوكر" أو رمز بديل بيعوض عن حروف مجهولة في أسماء الملفات.
• يعني إيه **Single-Character** ؟ معناه إنه مفروض يعوض عن حرف واحد فقط لا غير، مش أكثر من حرف.

- **\$** Variable substitution (contents of a variable).
- **\$** end-of-line. In a regular expression, a "\$" addresses the end of a line of text.
- **\$?** exit status variable. The \$? variable holds the exit status of a command, a function, or of the script itself.
- **\$\$** process ID variable. The \$\$ variable holds the process ID of the script in which it appears.
- **()** command group. (a=hello; echo \$a) > (ls; pwd; echo "Done")
- **()** array initialization. Array=(element1 element2 element3)
- تعبير القوس
- **{xxx,yyy,zzz,...}** Brace expansion. > mkdir {html,css,js}_project

```
[root@Lab01 karim]# touch {file1,file2,file3,file4,file5}
[root@Lab01 karim]# ls
file1 file2 file3 file4 file5
[root@Lab01 karim]#
```

3. متغير حالة الخروج (`?`) (Exit Status Variable)

أ. مع الأوامر العادية (`Command`):

- لفحص نتيجة تنفيذ آخر أمر تم كتابته في التيرمينال (هل تم بنجاح `0` أم حدث خطأ):

```
ls /not_found_file  
? echo
```

ب. مع الدوال (`Function`):

- لو عندك دالة داخل سكريبت وتحدد رقم خروج معين عند انتهائها:

```
check_user() {  
    id $1 &>/dev/null  
    return 5  
}  
  
check_user root  
echo $?
```

Bash

ج. مع السكريبت نفسه (`The Script Itself`):

- لمعرفة كود الخروج النهائي الذي أنهى به السكريبت تنفيذه:

```
myscript.sh/.  
? echo
```

ملحوظة توضيحية:

- إذا طبع لك رقم `0` ، فهذا معناه أن كل شيء تم بنجاح بدون أخطاء (`Success`).
- إذا طبع لك أي رقم ثاني من `1` إلى `255` ، فهذا معناه أن هناك مشكلة أو خطأ حدث أثناء تنفيذ الأمر (`Error`).

- **{a..z}** Extended Brace expansion.

```
[root@Lab01 karim]# echo {a..z}
a b c d e f g h i j k l m n o p q r s t u v w x y z
[root@Lab01 karim]# echo {1..20}
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
[root@Lab01 karim]#
```

- **{ } \;** pathname. Mostly used in find constructs.

```
[root@Lab01 karim]# ls
file1 file2 file3 file4 file5
[root@Lab01 karim]# find . -name file1 -exec rm -f {} \;
[root@Lab01 karim]# ls
file2 file3 file4 file5
[root@Lab01 karim]#
```

- **[]** Test expression between [].
- **[[]]** Test expression between [[]]. More flexible than the single-bracket [] test.

```
[root@Lab01 karim]# x=5;y=10
[root@Lab01 karim]# [[ $x -lt $y ]] && echo $y
10
[root@Lab01 karim]# [[ $x -gt $y ]] && echo $y
[root@Lab01 karim]#
```

اللي بيستخدم كبديل لاسم الملف هو العلامة `{ }` لوحدها فقط لا غير!

أما `/` وال `;` اللي في الآخر `( \ ; )`:

- `\` ال (Backslash) دي عشان تعمل Escape (إلغاء المعنى الخاص) لعلامة الفصلة المنقوطة، لأن الباش بيعتبر ال `;` نهاية لأي أمر عادي، فبنحط قبلها `\` عشان تتمرر كجزء من خيارات أمر ال `find`.
- `;` ال (الفصلة المنقوطة) دي إلزامية في نهاية خيار `exec-` عشان تقفل وتنهي الأمر اللي هيتنفذ على الملفات اللي تم إيجادها.

إذن، `{ }` هي فقط بديل اسم الملف، وباقي الرمز `\ ;` هو نظام كتابة إلزامي لنهاية أمر ال `find` !

ملحوظة توضيحية لربط الأوامر:

علامتي `ال &&` تستخدم لربط بين أمرين، ووظيفتها تنفيذ الأمر الثاني بشرط رجوع الأمر الأول، يعني إذا كان كود الخروج للأمر الأول يساوي صفر تماماً.

ملحوظة توضيحية للأقواس:

الأقواس المزدوجة `[ ]` هي أحسن من الأقواس الفردية `[ ]` لأنها بتتعامل مع المتغيرات الفارغة أو التي تحتوي على مسافات بشكل صحيح دون ما يحصل `syntax error` ، ومسموح بداخلها باستخدام علامات `ال &` وال `|` وكده يعني والكلام ده كله بدون الحاجة إن أنت تعمل قبلها `\` (على عكس الفردية).

➤ `>` `&>` `>&` `>>&` `<` redirection.

scriptname `>` filename redirects the output of scriptname to file filename.

Overwrite filename if it already exists.

command `&>` filename redirects both the stdout and the stderr of command to filename.

command `2>&1` redirects stdout of command to stderr.

scriptname `>>` filename appends the output of scriptname to file filename.

If filename does not already exist, it is created.

➤ | [pipe] Passes the output (stdout of a previous command to the input (stdin) of the next one, or to the shell.

أمر **sh**: يستقبل النص ده اللي هو **ls -l** ويعتبره كأنه أمر شل حقيقي وينفذه
الـ **sh** دي اختصار لـ **Bourne Shell** ، وهي عبارة عن مترجم أو مفسر للأوامر

```
[root@Lab01 karim]# echo ls -l
ls -l
[root@Lab01 karim]# echo ls -l | sh
total 0
-rw-r--r-- 1 root root 0 Dec 22 21:51 file2
-rw-r--r-- 1 root root 0 Dec 22 21:51 file3
-rw-r--r-- 1 root root 0 Dec 22 21:51 file4
-rw-r--r-- 1 root root 0 Dec 22 21:51 file5
```

```
[root@Lab01 karim]# ls -l | tr 'a-z' 'A-Z'
TOTAL 0
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE2
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE3
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE4
-RW-R--R-- 1 R00T R00T 0 DEC 22 21:51 FILE5
```

```
ls -l ثم cd /etc/rc.d/rc5.d
```

- هذا المجلد فيه مجموعة من ال `symlink` التي بتشير لمجموعة من ال `system shell script` بيعملها `execute` أثناء ال `startup` عشان يقوم بعض ال `services`, زي مثلاً سكربت ال `network` وهو مسؤول عن ال `network interface` سواء `up` أو `down`.

```
less ../.nti.d/network
```

 (معرفة كيفية كتابة ال `scripts` من الداخل)

`Command Interpreter` (تحديد المفسر)

- يجب دائماً تحديد ال `Interpreter` في بداية السكريبت، وال `Default shell` عندنا هو ال `bash`, ولو كتبت أوامر بلغة ثانية (مثل بايثون) ولم تحدد المفسر أو لم تحدد ال `interpreter` الخاص بها سيحدث `error`, ولذلك يجب كتابة المسار الخاص به.

طرق تنفيذ ال `Shell Script`:

- الطريقة الأولى: كتابة اسم ال `interpreter` ثم كتابة اسم المسار الكامل للسكريبت (مثل: `python /path/to/script.py`), أو استخدام ال `./` لو استخدام محلي مع مسار السكريبت.
- الطريقة الثانية: لتشغيل السكريبت من أي مكان بمجرد كتابة اسمه فقط, قم بوضعه داخل أي `location` من ال `PATH variable`.

خطوات إنشاء وتنفيذ سكريبتات لغات مختلفة (Bash, Python, Perl)

1. إنشاء الملفات

الأمر: `touch hello.sh hello.py hello.pl`

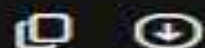
ملحوظة: الامتدادات هنا (`.sh` , `.py` , `.pl`) لا تؤثر جوهرياً على عمل النظام في التنفيذ، ولكنها لوصف وتوضيح نوع الملف فقط.

2. معرفة مسار الباش

الأمر: `which bash`

3. كتابة محتوى السكريبتات

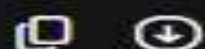
أمر ال Bash : `( vim hello.sh )`



Bash

```
#!/bin/bash
echo "Hello World"
```

أمر ال Python : `( vim hello.py )`



Python

```
#!/usr/bin/python
print("Hello World")
```

○ أمر ال Perl (vim hello.pl) :



Perl

```
#!/usr/bin/perl  
print "Hello World\n";
```

4. إعطاء الصلاحيات وتنفيذ السكريبتات

○ الأمر: `chmod +x hello.sh hello.py hello.pl`

○ السبب: إعطاء صلاحية التنفيذ (Executable Permission) للملفات حتى يسمح للنظام بتشغيلها.

○ التنفيذ المباشر (مجموعين معاً):



Bash

```
./hello.sh && ./hello.py && ./hello.pl
```

○ التنفيذ باستخدام المفسر صراحة:



Bash

```
bash hello.sh && python hello.py && perl hello.pl
```


5. اختبار إزالة سطر الـ Shebang في بايثون

- الأمر: `vim hello.py` (ثم حذف السطر الأول الذي يحتوي على `#!/usr/bin/python`).

- التنفيذ: `hello.py/.`

- النتيجة: سيعطي النظام `Syntax Error` لأن الـ Shell لن يعرف المفسر المناسب لتنفيذ الكود لعدم وجود الـ Shebang.

6. التنفيذ بمسار مطلق ونقل الملفات لمتغير الـ PATH

- تنفيذ بمسار مطلق: `root/hello.pl/.` (أو المسار الفعلي للملف).

- عرض مسارات الـ PATH: `echo $PATH`

- نقل السكريبتات إلى مسار عام: `mv hello.* /usr/local/bin`

- الفائدة: بعد نقلها لهذا المجلد المعرف في الـ PATH، يمكنك كتابة اسم السكريبت مباشرة (`hello.sh`) من أي مكان في النظام وسيعمل فوراً دون الحاجة لكتابة مساره أو استخدام `./`.

Create script backup /etc daily on the backup server

```
touch backup.sh
```

```
vim backup.sh
```



Bash

```
#!/bin/bash
```

```
# Tar the /etc directory
```

```
rar -cf etc_backup.tar /etc
```

```
# Transfer the tar archive to the backup server using scp command
```

```
scp etc_backup.tar root@192.168.1.10:/backup
```

```
-P 22
```

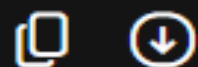
```
# Clean up: remove the local tar archive
```

```
rm -rf etc_backup.tar
```

```
chmod +x backup.sh
```

```
backup.sh/.
```


○ للتأكد أن الملف وصل على السيرفر البعيد:



Bash

```
cd /backup  
ls
```

ملحوظات وتحسينات هامة للسكريبت:

- **جعل السكريبت ذكياً (Smart enough)**: السكريبت الحالي ليس به أي Logic للتحقق من الأخطاء، مما قد يسبب Unexpected behavior إذا حدث خطأ وتابع التنفيذ.
- **التحقق من المجلدات والاتصال**: يفضل استخدام شروط الـ (if) للتحقق من وجود المجلدات (سواء المحلية أو على السيرفر البعيد) قبل تنفيذ النقل لتفادي أي Error .
- **التعامل مع الأخطاء**: في حالة حدوث مشاكل في الاتصال أو فشل أمر الـ scp ، يُفضل الخروج الفوري من السكريبت أو طباعة Debug log لمعرفة سبب المشكلة بدلاً من الاستمرار في الحذف (rm -rf).

Quotes in Shell Scripting

Single Quotes (' ' - Full Quoting) ○

○ تقوم بـ Disable (تعطيل) المعنى الخاص لأي حرف داخلها (Special Meaning). الحرف بيكتب وينطبع زي ما هو بحرفيته تماماً من غير ما الـ Shell يفسره أو يعوض قيمته (مثل علامة \$).

Double Quotes (" " - Partial Quoting) ○

○ تسمح بتفسير بعض الحروف الخاصة مثل الـ Variable Substitution والـ Command Substitution. بمعنى أنها تحافظ على المعنى الخاص لعلامة الدولار (\$) لتستبدل المتغير بقيمته، بينما تعطل معاني أخرى.

أمثلة عملية وتطبيقية على المتغيرات (Variables)

hostname ○

hostname -s ○

○ إسناد قيمة لأمر داخل متغير (طريقتان متطابقتان):



Bash

```
HOST=$(hostname -s) == HOST=`hostname -s`
```

○ المغزى: الطريقتان تقومان بتنفيذ الأمر بداخلهما وتخزين النتيجة (Output) داخل المتغير HOST.

○ استخدام علامة الدولار (\$) :

○ `echo $HOST`

○ المغزى: علامة الـ \$ هنا لها Special Meaning ، وهي أن النظام يفهم أنك تريد استرجاع وعرض القيمة المخزنة داخل الـ variable الذي اسمه HOST .

○ توضيح استخدام Double Quotes مع المتغيرات:



Bash

```
echo "your user name variable is $USER"
```

○ المغزى: الـ Double Quotes هنا لم تقم بعمل تعطيل (Disable) لعلامة الدولار (\$) . وذلك لأن الـ Double Quotes تسمح بالـ Variable Substitution (استبدال المتغير بقيمته) ، مما يتيح لك دمج المتغيرات داخل النصوص وطباعتها بشكل طبيعي ، فسيظهر اسم المستخدم الحقيقي مكان \$USER .

○ الهروب باستخدام علامة الـ (\) :



Bash

```
echo "\"Hello, world\"" == echo "'hello, world'"
```

○ المغزى: طريقتان للحصول على نفس النتيجة وهي طباعة النص بداخله علامات تنصيص ظاهرية ، إما باستخدام الـ Escape character (\) داخل الـ Double quotes ، أو وضع الـ Double quotes داخل الـ Single quotes .

Files and Pathnames with Multiple Spaces

- إنشاء مجلد بمسافات (طرق متطابقة):

```
mkdir "mahara tech" == 'mahara tech' == mahara\ tech
```

- ملاحظة: هذه الطريقة غير مفضلة (not recommended) عموماً عند تسمية الملفات والمجلدات لتجنب المشاكل.

- المشكلة عند الدخول بدون حماية المسافات:

```
cd /root/mahara tech
```

- النتيجة: سيظهر error لأن النظام سيعتبر كل كلمة وسيطاً مستقلاً وسيخبرك بـ .too many arguments

- الحل السريع أثناء الكتابة:

- يمكنك كتابة أول حروف من الكلمة (مثل mahara) ثم الضغط على زر Tab في لوحة المفاتيح، وسيقوم النظام تلقائياً بإكمال الاسم وحماية المسافات نيابة عنك.

العمليات الحسابية البسيطة والمتغيرات (Arithmetic Operations)

- إسناد القيم، الجمع، الضرب، وتعطيل المعاني الخاصة (مجمعة في شاشة واحدة):

```
# Assign values and echo variables
x=5
y=10
echo $x $y

# Addition inside square brackets
echo x + y = ${x+y}

# Disable special meaning for $
echo x + y = \${x+y}

# Using double quotes with arithmetic operations
echo "x + y = ${x+y}"

# Multiplication and handling asterisk (*) special meaning
echo x * y = ${x*y}
echo x \* y = ${x*y}

# Display the Process ID (PID) of the current running script
echo $$
```

- المغزى لعلامة الـ *asterisk* (*) :
- داخل الأقواس المربعة (\$ [...]) : تعني عملية الضرب (*multiplication*).
- خارج الأقواس (في الـ *wildcard representation*) : تعني مطابقة أي عدد من الحروف (صفر أو أكثر في الـ *regular expression*) ، أو استخدامها مفردة لتعني سرد وعرض جميع الملفات والمجلدات (*list all files and directory*) الموجودة في المجلد الحالي.
- المغزى لـ (\$ \$) : تستخدم لطباعة وعرض الـ *PID* (Process ID) الخاص بالسكربت أو الشل الحالي أثناء تنفيذه.

Running commands more efficiently using loops

Operations on code blocks are the key to structured and organized shell scripts. Looping and branching constructs provide the tools for accomplishing this.

- مفهوم ال Loops: استخدام التكرار لتنفيذ الأوامر بكفاءة عالية وبدون تكرار كتابة الكود يدويًا.
- أهمية تنظيم الكود (Code Blocks): تنفيذ العمليات داخل كتل برمجية منظمة يعتبر المفتاح الأساسي لكتابة شل سكريبت (Shell Scripts) احترافي وسهل الصيانة.
- الأدوات المستخدمة: استخدام أدوات ال Loop وال Branching يوفر القدرة الكاملة للتحكم في تدفق تنفيذ الأوامر داخل النظام.

الشرح: ال Loop بتعدي على قائمة عناصر (list) عنصراً بعنصر، وكل عنصر بيتاخذ يتحط جوه المتغير arg عشان يتنفذ عليه مجموعة الأوامر جوه بلوك ال do ... done.

Loops

- A loop is a block of code that iterates a list of commands as long as the loop control condition is true.

- for loop

```
for arg in [list]
do
```

```
command1
command2
command3
```

```
done
```

```
[root@Lab01 karim]# NUMBERS="1 2 3 4 5"
[root@Lab01 karim]# for number in $NUMBERS
> do
> echo -n "$number"
> done
12345[root@Lab01 karim]#
```

```
[root@Lab01 karim]# for file in file*
> do
> ls -l $file
> done
-rw-r--r--. 1 root root 0 Dec 26 11:23 file1
-rw-r--r--. 1 root root 0 Dec 26 11:23 file2
-rw-r--r--. 1 root root 0 Dec 26 11:23 file3
-rw-r--r--. 1 root root 0 Dec 26 11:23 file4
-rw-r--r--. 1 root root 0 Dec 26 11:23 file5
```

2. المثال الأول: حلقة تكرار على أرقام (Numbers Loop)



Bash

```
NUMBERS="1 2 3 4 5"  
for number in $NUMBERS  
do  
    echo -n "$number"  
done
```

- إيه اللي بيحصل هنا؟
 - بنخزن مجموعة أرقام جوه المتغير `NUMBERS`.
 - ال `for` يتاخد كل رقم لوحده وتخزنه في متغير اسمه `number`.
 - أمر `echo -n` بيطبع الرقم من غير ما ينزل سطر جديد (عشان الأرقام تلتزق جنب بعضها).
 - النتيجة اللي بتظهر على الشاشة: `12345`.

3. المثال الثاني: حلقة تكرار على ملفات باستخدام Wildcard (Files Loop)



Bash

```
for file in file*  
do  
    ls -l $file  
done
```

- إيه اللي بيحصل هنا؟
 - بنقول لل `for` ابحتي عن أي ملف يبدأ اسم بكلمة `file` (عن طريق `*file`).
 - ال `Loop` هتمسك ملف ملف، وتنفذ عليه أمر `ls -l` عشان تعرض تفاصيل وصلاحيات كل ملف على حدة (`file1` , `file2` , وهكذا).

1. إنشاء ملفات تلقائياً



Bash

```
for i in 1 2 3 4 5
do
    touch file_$i
    ls
    sleep 2
done
```

- الوظيفة: ينشئ 5 ملفات تدريجياً (file_1 إلى file_5) ويعرضها مع انتظار ثانيتين في كل لفة.

2. إعادة تسمية الملفات (Batch Renaming)



Bash

```
for file in file*
do
    mv $file New_$file
    ls New*
    sleep 2
done
```

- الوظيفة: يبحث عن الملفات التي تبدأ بـ file ويعيد تسميتها لتصبح مسبوقه بكلمة _New .

Numerical Comparison Operators

(معاملات المقارنة العددية)

المعامل (Operator)	معناه باللغة الإنجليزية	مثال للتوضيح
eq-	Equal to	[i -eq 10\$] (هل المتغير i\$ يساوي 10؟)
ne-	Not equal to	[i -ne 10\$] (هل المتغير i\$ لا يساوي 10؟)
gt-	Greater than	[i -gt 10\$] (هل المتغير i\$ أكبر من 10؟)
ge-	Greater than or equal to	[i -ge 10\$] (هل المتغير i\$ أكبر من أو يساوي 10؟)
lt-	Less than	[i -lt 10\$] (هل المتغير i\$ أقل من 10؟)
le-	Less than or equal to	[i -le 10\$] (هل المتغير i\$ أقل من أو يساوي 10؟)

1. مثال ال While Loop (الطباعة حتى الرقم 10)



Bash

```
i=1
while [ $i -le 10 ]
do
    echo $i
    let i+=1
done
```

- **الوظيفة:** تطبع الأرقام من 1 إلى 10 طالما أن المتغير أقل من أو يساوي 10 (le-). مع زيادة القيمة بوحدة في كل خطوة.

2. مثال ال Until Loop (حلقة التكرار العكسية حتى يصبح الشرط صحيحاً)



Bash

```
i=1
until [ $i -gt 10 ]
do
    echo $i
    let i+=1
done
```

- **الوظيفة:** تستمر في التكرار وطباعة الأرقام طالما أن الشرط خطأ (i\$ ليس أكبر من 10)، وتتوقف فوراً عندما يصبح الشرط صحيحاً.

أولاً: مفاهيم أساسية (Condition, Exit Status, Semicolon) وملحوظة المسافات)

1. هل أي حاجة جوه [] بتعتبر Condition؟

- نعم تماماً! القوسين المربعين [] (أو القوسين المزدوجين [[]]) هما المكان المخصص لكتابة الشروط (Conditions) في اللينكس.
- نتيجتها دائماً بتكون إما True (صحيح) أو False (خطأ).

2. ملحوظة ذهبية هامة جداً (Space Rule):

- لا بد أن تترك مسافة (Space) بعد القوس الفاتح وقبل القوس القافل داخل [...] (وكذلك حول معامل المقارنة)، لأنك لو كتبتها لازقة في بعضها (زي ما ظهر في الخطأ -'bash: [: missing ') هيطلع لك إرور فوراً لأن اللينكس بيتعامل مع الأقواس كأوامر وليست كرموز شروط.

3. ليه نتيجة ال True بتطلع 0 وال False بتطلع رقم ثاني (مثل 1)؟

- في عالم اللينكس وبخلاف لغات البرمجة الثانية، ال 0 يعني نجاح العملية (Success / True)، بينما أي رقم ثاني (زي 1 أو غيره) بيعبر عن وجود خطأ أو فشل (Failure / False). ولما بنكتب `echo $?` إحنا بنطلب من التيرمينال تطبع نتيجة كود الخروج (Exit Status) لآخر أمر اتنفذ.

4. إيه معنى ال Semicolon (فاصلة منقوطة ؛) بين الأوامر؟

- بتخليك تكتب أمرين أو أكثر ورا بعض في سطر واحد.
- هل هي متوقفة على بعض؟ لا خالص! ال ؛ بتنفذ الأمر الأول بغض النظر نجح أو فشل، وبعدين تنفذ الأمر الثاني وراه مباشرة على الترتيب من الشمال لليمين.

ثانياً: ملخص أنواع الـ Loops (للسلايدز والشرح)

- الـ `for loop`: تُستخدم عندما يكون عدد مرات التكرار معروفاً ومحدد مسبقاً (على عناصر معينة أو قائمة).
- الـ `while loop`: تستمر في التكرار طالما أن الشرط `true` (صحيح).
- الـ `until loop`: عكس الـ `while`. تستمر في التكرار طالما أن الشرط `false` (خطأ) وتتوقف عندما يصبح صحيحاً.

ثالثاً: معاملات مقارنة النصوص (String Comparison Operators)

بجانب المعاملات العددية، فيه معاملات مخصصة لمقارنة النصوص (Strings) وتستخدم عادة مع الأقواس المزدوجة `[[ ]]`:

المعامل (Operator)	معناه باللغة الإنجليزية	مثال للتوضيح
<code>=</code> أو <code>==</code>	Equal to (النصوص متطابقة)	<code>[["abc" = "abc"]]</code> (هل النصان متطابقان؟)
<code>!=</code>	Not equal to (النصوص غير متطابقة)	<code>[["abc" != "def"]]</code> (هل النصان مختلفان؟)

```
# 1. مقارنة عددية صحيحة (النتيجة تخرج 0)
[ 1 -eq 1 ]; echo $?

# 2. مقارنة عددية خاطئة (النتيجة تخرج 1)
[ 1 -ne 1 ]; echo $?

# 3. اختيار هل 8 أكبر من 2 (صحيح -> 0)
[ 8 -gt 2 ]; echo $?

# 4. خطأ مقصود لعدم ترك مسافة قبل الأقواس الأخير (Missing ')
[ 8 -ge 8 ]; echo $?

# 5. التصحيح: اختيار هل 8 أكبر من أو يساوي 8 مع مراعاة المسافات (صحيح -> 0)
[ 8 -ge 8 ]; echo $?

# 6. اختيار هل 8 أقل من 8 (خاطئ -> 1)
[ 8 -lt 8 ]; echo $?

# 7. اختيار هل 8 أقل من أو يساوي 8 (صحيح -> 0)
[ 8 -le 8 ]; echo $?

# 8. مقارنةصوص متطابقة باستخدام الأقواس المزدوجة (صحيح -> 0)
[[ "abc" = "abc" ]]; echo $?

# 9. مقارنةصوص متطابقة بمدخلات مختلفة (خطأ لأنهم مش زي بعض -> 1)
[[ "abc" = "def" ]]; echo $?

# 10. اختيار عدم تطابق نصين باستخدام != (صحيح لأنهم مختلفين -> 0)
[[ "abc" != "def" ]]; echo $?
```

Unary Operators (العوامل الأحادية)

المعامل	معناه باللغة الإنجليزية	الشرح بالعربية
<code>z-</code>	String is empty	فحص هل النص فارغ أم لا
<code>n-</code>	String is not empty	فحص هل النص غير فارغ (يحتوي على حروف)
<code>d-</code>	Directory exists	فحص هل المسار مجلد موجود
<code>f-</code>	File exists and is regular	فحص هل الملف موجود وعادي
<code>e-</code>	File or directory exists	فحص هل الملف أو المجلد موجود بأي صفة
<code>s-</code>	File is not empty	فحص هل الملف موجود وحجمه أكبر من صفر (ليس فارغاً)
<code>r / -w / -x-</code>	File is readable / writable / executable	فحص صلاحيات الملف (القراءة / الكتابة / التنفيذ)


```

# 1. فحص تمس قارغ (النتيجة 0 لأن المتغير قارغ بالفعل)
STRING=""
[ -z "$STRING" ]; echo $?

# 2. فحص تمس يحتوي على قيمة هل هو قارغ (النتيجة 1 لأنه تم قارغ)
STRING='abc'
[ -z "$STRING" ]; echo $?

# 3. فحص (النتيجة 0 لأنه يحتوي على) فحص هل النص غير قارغ
[ -n "$STRING" ]; echo $?

# 4. عبارة عن مجلد (النتيجة 0 لأنه موجود) /etc فحص هل المسار
[ -d /etc ]; echo $?

# 5. فحص مجلد غير موجود (النتيجة 1 لأن المجلد تم موجود)
[ -d /etc1 ]; echo $?

# 6. موجود وعادي (النتيجة 0) passwd فحص هل ملف
[ -f /etc/passwd ]; echo $?

# 7. فحص ملف غير موجود (النتيجة 1)
[ -f /etc/passwd1 ]; echo $?

# 8. فحص صلاحية القراءة للملف (النتيجة 0 لأن مسجوح قراءته)
[ -r /etc/passwd ]; echo $?

# 9. عرض صلاحيات الملف وتفاصيله بالتفصيل
ls -l /etc/passwd

# 10. فحص صلاحية التنفيذ للملف (النتيجة 1 لأنه ليس ملف تنفيذي)
[ -x /etc/passwd ]; echo $?

```

Exit and exit status

- The **exit** command may be used to terminate a script, just as in a C program. It can also return a value, which is available to the script's parent process.
- Every command returns an exit status (sometimes referred to as a return status or exit code). A successful command returns a **0**, while an unsuccessful one returns a **non-zero** value that usually may be interpreted as an error code.
- When a script ends with an exit that has no parameter, the exit status of the script is the exit status of the last command executed in the script (previous to the exit).

```
#!/bin/bash

COMMAND_1

. . .

# Will exit with status of last command.
COMMAND_LAST

exit
```

```
#!/bin/bash

COMMAND_1

. . .

# Will exit with status of last command.
COMMAND_LAST

exit $?
```

Exit and Exit Status

- وظيفة أمر `exit`: تُستخدم لإنهاء السكريبت فوراً وترجع قيمة الـ Exit Status للـ CLI.
- ملحوظة عن الصورتين: الـ `exit` في الصورة الأولى والـ `exit $?` في الصورة الثانية تعطي نفس النتيجة تماماً، لأنها تمرر كود خروج آخر أمر بشكل صريح.

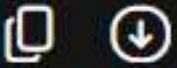
تجارب عملية مقترحة

- التجربة الأولى: افتح سكريبت واكتب في آخره أي أمر خطأ (مش موجود) ثم شغله؛ النتيجة: السكريبت سيعطي خطأ وعند كتابة `echo $?` ستجد رقم غير الصفر يدل على الفشل.
- التجربة الثانية: جرب تفتح نفس السكريبت وتضع كلمة `exit` قبل الأوامر؛ النتيجة: سيتوقف السكريبت عندها فوراً ومش هيوصل للأمر خالص.
- التجربة الثالثة: جرب تكتب `exit 10` في نهاية الأوامر؛ النتيجة: فور خروج السكريبت اكتب `echo $?` هتلاقي التيرمينال رد عليك بالرقم 10 اللي انت حددته بنفسك.

Testing script Inputs

Every reasonably complete programming language can test for a condition, then act according to the result of the test. Bash has the test command, various bracket and parenthesis operators, and the if/then construct.

أولاً: شرح مثال الـ If / Else (للمقارنة العددية)



Bash

```
x=5
y=10
if [ $x -gt $y ]
then
    echo "X is greater than y"
else
    echo "X is lower than y"
fi
```

الوظيفة: بنحزن قيم في متغيرين (`x=5` و `y=10`)، ونستخدم جملة الشرط `if` مع معامل المقارنة `gt-` لنفحص هل `x` أكبر من `y` ؟ وبما أن الشرط غير صحيح، ينفذ الـ `else` ويطبع على الشاشة `.X is lower than y`.

Activat

ثانياً: تفاصيل أمر ال `cmp` (مقارنة الملفات)

1. إيه هو أمر ال `cmp` ؟

○ هو أمر يقارن بين ملفين بايت ب بايت (Byte by Byte).

2. ال (Return (Exit Status) بيطلع كام؟

○ لو الملفين متطابقين تماماً: أمر ال `cmp` بيرجع 0 (يعني نجاح / True).

○ لو الملفين مختلفين: بيرجع 1 (يعني اختلاف / False) وبيتطبع على الشاشة أول مكان حصل فيه الاختلاف (مثل رقم البايت ورقم السطر).

3. إزاي ما بيطلعش حاجة في ال Output وبيعتمد على ال Return؟

○ لما بنكتب `dev/null/<&` جنب الأمر، إحنا بنقول لل Terminal: "خد أي كلام بيطلع على الشاشة واقرمه في سلة المهملات (اكتبه في ال null)", فمش بيظهر أي كلام قدامنا على الشاشة، وبنكتفي تماماً بكود الخروج (`$?`) أو استخدام الأمر جوه ال `if` مباشرة عشان يعرف هل حصل تطابق ولا اختلاف بناءً على ال 0 أو ال 1).

4. هل شرط استخدام ال `if` يكون معاها `[ ]` ؟

○ لا مش شرط خالص! ال `if` في الباش مش بتدور على أقواس مربعة وبس، ال `if` بتنفذ أي أمر عادي. لو أمر ال `cmp` نجح ورجع 0 (يعني الملفين زي بعض)، ال `if` بتعتبر إن الأمرين نجحوا وتنفذ الفرع الأول (`then`)، ولو رجع 1 بتروح لل `else`. زي ما ظهر في آخر مثالين في الصورة لما استخدمنا `if cmp ...` بدون أقواس مربعة نهائياً.

1. مقارنة الملفات باستخدام `cmp` والتحقق من النتيجة

Bash

```
# مقارنة ملفين متطابقين تماماً (النتيجة 0 يعني متشابهين)
cmp /etc/passwd /etc/passwd
echo $?

# مقارنة ملفين مختلفين (النتيجة 1 ويظهر مكان الاختلاف)
cmp /etc/passwd /etc/shadow
echo $?

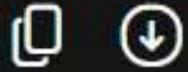
# إبقاء المخرجات بـ if داخل شرط cmp استخدام أمر /dev/null
if cmp /etc/passwd /etc/passwd &>/dev/null
then
    echo "both files are matched"
else
    echo "both files are different"
fi
```

الشرح: أمر الـ `cmp` يقارن بين ملفين. وإذا وضعنا `dev/null/<&` فهذا يخفي أي رسائل تظهر على الشاشة وتكتفي بكود الخروج (0 للتطابق و 1 للاختلاف) لتوجيه الـ `if`.

ملحوظة هامة جداً قبل الانتقال لفحص الخدمات:

- كلمة `fi` في نهاية الجملة تُستخدم لكي يفهم نظام اللينكس أن الشرط قد انتهى (وهي عكس `if`).
- لاحظ جيداً أن كلمة `else` لا يأتي بعدها كلمة `then` أبداً (الـ `then` تكون فقط مع الـ `if`).

2. فحص حالة خدمة (Service Status) وكود الخروج الخاص بها



Bash

فحص حالة خدمة معينة هل هي شغالة ولا لا

```
systemctl status psacct.service
```

(active) التحقق السريع من حالة الخدمة هل هي نشطة

```
systemctl is-active psacct.service
```

(inactive/يعطي رقم 3 لأن الخدمة غير نشطة) طباعة كود الخروج لآخر أمر

```
echo $?
```

- الشرح: أمر `systemctl is-active` يخبرنا بحالة الخدمة، وكود الخروج (`$?`) هنا يعطي رقم 3 لأن الخدمة ليست في حالة تشغيل (inactive).

3. استخدام الشرط لتشغيل أو إيقاف الخدمة بناءً على حالتها



Bash

```
# إذا كان كود الخروج لا يساوي صفر (يعني الخدمة مش هتالة). قم بتشغيلها. وإلا أوقفها
if [ $? -ne 0 ]
then
    systemctl start psacct.service
else
    systemctl stop psacct.service
fi

# لتكون واضحة fi والـ else تقسيم الـ (كتابة الشرط في سطرين مريحين للشاشة)
if [ $? -ne 0 ]; then systemctl start psacct.service
else systemctl stop psacct.service; fi

# بدون الحاجة لتعقيدات سابقة if التحقق المباشر من حالة الخدمة داخل جملة الـ
if systemctl is-active psacct.service &>/dev/null
then
    systemctl start psacct.service
else
    systemctl stop psacct.service
fi
```

- الشرح: هنا يستخدم نتيجة الأمر السابق (`$? -ne 0`) أو بنفحص حالة الخدمة مباشرة بـ `if` عشان لو مش شغالة نعمل لها `start` ولو شغالة نعمل لها `stop`.

Test constructs

- An **if/ then** construct tests whether the exit status of a list of commands is 0 (since 0 means "success" by UNIX convention), and if so, executes one or more commands.
- There exists a dedicated command called **[** (left bracket special character). It is a synonym for **test**, and a built-in for efficiency reasons. This command considers its arguments as comparison expressions or file tests and returns an exit status corresponding to the result of the comparison (0 for true, 1 for false).
- Bash introduced the **[[...]]** extended test command, which performs comparisons in a manner more familiar to programmers from other languages. Note that **[[** is a keyword, not a command.

Bash sees `[[ $a -lt $b ]]` as a single element, which returns an exit status.

```
[root@Lab01 karim]# x=5;y=10
[root@Lab01 karim]# [[ $x -lt $y ]] && echo $?
0
[root@Lab01 karim]# [[ $y -lt $x ]] || echo $?
1
```

شرح بنى الاختبار (Test Constructs)

- **النقطة الأولى (دور ال if/then):** جملة ال `if/then` تفحص كود الخروج لمجموعة أوامر، ولو طلع ال Exit Status يساوي `0` (واللي معناه "نجاح" حسب قواعد يونكس)، تبدأ في تنفيذ الأوامر اللي جوه ال `then`.
- **النقطة الثانية (القوس المفرد كأمر مستقل):** القوس الأيسر `[` هو أمر خاص ومستقل يعتبر مرادفاً تماماً لأمر ال `test`، وتم بناؤه جوه النظام كأمر أساسي (Built-in) ليكون أسرع وأكثر كفاءة، حيث يتعامل مع ما بداخله كتعبيرات مقارنة ويرجع `0` لو صح و `1` لو غلط.
- مثال عملي على القوس المفرد `[ ]`:



Bash

```
if [ 5 -gt 3 ]; then echo "Yes, greater"; fi
```

- **النقطة الثالثة (الأقواس المزدوجة المحسنة):** طور الباش أمر الاختبار وأدخل الأقواس المزدوجة `[[ ... ]]` لتكون المقارنات مألوفة أكثر للمبرمجين، وخذ بالك إن `[]` تعتبر كلمة مفتاحية (Keyword) وليست أمراً مستقلاً.
- **النقطة الرابعة (التعامل كوحدة واحدة):** يتعامل الباش مع `[[ a -lt $b$ ]]` كقطعة واحدة متكاملة تعطي في النهاية كود خروج (Exit status) مباشر.

مميزات الأقواس المزدوجة [[]] مقارنة بالقوس الفردي [] :

1. حماية المتغيرات (Variable Protection / Word Splitting)



Bash

الحالة الأولى (قوس مفرد بدون علامات تنقيص - سيحدث خطأ ولن يكتمل السكريبت)

```
name=""
```

```
if [ $name = "admin" ]; then echo "Match"; fi
```

الحالة الثانية (قوس مفرد مع علامات التنقيص للحماية)

```
name=""
```

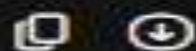
```
if [ "$name" = "admin" ]; then echo "Match"; fi
```

الحالة الثالثة (أقواس مزدوجة آمنة تماماً)

```
name=""
```

```
if [[ $name == "admin" ]]; then echo "Match"; fi
```


2. استخدام العمليات المنطقية (Logical Operators)



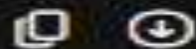
Bash

```
# المثال الأول (خطأ: استخدام الرموز المباشرة && داخل القوس الفردي):
x=5; y=10
if [ $x -gt 0 && $y -lt 20 ]; then echo "Valid"; fi

# (داخل القوس الفردي 0 و -0 مع: استخدام المعاملات) المثال الثاني:
x=5; y=10
if [ $x -gt 0 -a $y -lt 20 ]; then echo "Valid"; fi

# المثال الثالث (مع: استخدام المعاملات المنطقية بكل سهولة داخل الأقواس المزدوجة):
x=5; y=10
if [[ $x -gt 0 && $y -lt 20 ]]; then echo "Valid"; fi
```

3. مطابقة النصوص باستخدام الويلد كارد ماسك (Pattern Matching / Wildcard)



Bash

```
# في حالة القوس الفردي (لا يفهم النتيجة قيمتهما حرفاً عادياً وتتشكل المقارنة):
filename="report.txt"
if [ $filename = *.txt ]; then echo "Match"; fi

# في حالة الأقواس المزدوجة (تفهم النتيجة وتقوم بمطابقة النصوص بأكفاء):
filename="report.txt"
if [[ $filename == *.txt ]]; then echo "Match"; fi
```

طرق فحص الشروط الثلاثة في الباش مع الـ `if`

○ الطريقة الأولى (أمر `test` الصريح):

○ `if test $x -eq 10; then echo "Equal"; fi` (فحص الشرط باستخدام كلمة `test` مباشرة).

○ الطريقة الثانية (الأقواس المربعة Bracket operators):

○ `if [ $x -eq 10 ]; then echo "Equal"; fi` (أو باستخدام الأقواس المزدوجة `[[ x -eq 10$ ]]` للفحص المتقدم).

○ الطريقة الثالثة (الأقواس الدائرية Parentheses للعمليات الحسابية):

○ `if (( x == 10 )); then echo "Equal"; fi` (فحص المقارنات الرياضية بالأقواس الدائرية).

ملاحظات هامة جداً لضبط القاعدة:

1. شرط الاستخدام (الأقواس الدائرية مقابل المربعة):

○ رموز المقارنة الرمزية (`<` , `>` , `<=` , `>=` , `==` , `!=`) تُستخدم حصرياً مع الأقواس الدائرية المزدوجة `(( ))` فقط، ولا تستخدم نهائياً مع الأقواس المربعة المفردة أو المزدوجة.

○ أما المعاملات الحرفية (`gt-` , `lt-` , `eq-` ، إلخ) فلا تُستخدم إلا مع الأقواس المربعة `[ ]` أو `[[ ]]`، ولا تعمل أبداً داخل الأقواس الدائرية.

2. طبيعة البيانات (أرقام فقط):

○ الأقواس الدائرية المزدوجة `(( ))` ومقارناتها الرياضية (`<` , `>` , `==` ، إلخ) مخصصة للأرقام فقط، ولا تُستخدم لمقارنة النصوص (Strings).

(()) Arithmetic Operators in Parentheses

المعامل

معناه باللغة الإنجليزية

مثال للتوضيح

((x == 5))

Equal to

==

((x != 5))

Not equal to

!=

((x > 5))

Greater than

>

((x >= 5))

Greater than or equal

>=

((x < 5))

Less than

<

((x <= 5))

Less than or equal

<=

Conditional structures

➤ There are various types of conditional statements in Bash

- if-else statement.
- if..elif..else statement.
- Nested if/ then condition.

▪ if-else statement

```
if [ condition-true ]
then
    command 1
    command 2
    ...
else
    # Optional (may be left out if not needed).
    # Adds default code block executing if original condition tests false.
    command 3
    command 4
    ...
fi
```

▪ if..elif..else statement

- ▣ **elif** is a contraction for else if. The effect is to nest an inner if/then construct within an outer one.

```
if [ condition1 ]  
then  
    command1  
    command2  
    command3  
elif [ condition2 ]  
# Same as else if  
then  
    command4  
    command5  
else  
    default-command  
fi
```

1. مثال على جملة if-else العادية:



Bash

```
x=10
if [ $x -gt 5 ]; then
    echo "الرقم أكبر من 5 (الشرط تحقق)"
else
    echo "الافتراضي else جزء الـ (الرقم ليس أكبر من 5)"
fi
```

2. مثال على جملة if..elif..else المتعددة الشروط:



Bash

```
score=85
if [ $score -ge 90 ]; then
    echo "(A) التقدير: ممتاز"
elif [ $score -ge 80 ]; then
    echo "(B) التقدير: جيد جداً"
elif [ $score -ge 70 ]; then
    echo "(C) التقدير: جيد"
else
    echo "التقدير: راسب / مقبول أقل"
fi
```


▪ Nested if/ then Condition

- Condition tests using the if/then construct may be nested. The net result is equivalent to using the && compound comparison operator above.

```
if [ condition1 ]  
then  
  if [ condition2 ]  
  then  
    do-something # But only if both "condition1" and "condition2" valid.  
  fi  
fi
```

- When **if** and then are on same line in a condition test, a **semicolon** must terminate the if statement. Both if and then are keywords. Keywords (or commands) begin statements, and before a new statement on the same line begins, the old one must terminate.

```
if [ -x "$filename" ]; then
```

1. مثال عملي على Nested if:



Bash

```
x=10; y=20
if [ $x -gt 5 ]; then
    if [ $y -gt 15 ]; then
        echo "(معادل استخدام &&)"
    fi
fi
```

2. شرح قاعدة ال Semicolon (الفاصلة المنقوطة):

عندما توضع كلمة `if` وكلمة `then` على نفس السطر في اختبار الشرط، يجب أن تنتهي جملة ال `if` بفاصلة منقوطة (`;`) لأن الباش يتطلب إنهاء الأمر القديم قبل بدء أمر جديد على نفس السطر.

○ مثال عملي:



Bash

```
if [ -x "$filename" ]; then echo "الملف قابل للتنفيذ"; fi
```

Regular expressions

- Some of the commonly used commands with Regular expressions are tr, sed, vi and grep. Listed below are some of the basic Regex.

Symbol	Descriptions
.	replaces any character
^	matches start of string
\$	matches end of string
*	matches up zero or more times the preceding character
\	Represent special characters
()	Groups regular expressions
?	Matches up exactly one character


```
echo "hello" | tr 'a-z' 'A-Z'
```

- لماذا تم وضعه مع ال Regular Expressions رغم أنه لا يدعمها بالكامل؟
لأن أمر ال `tr` يتعامل مع "نطاقات الحروف البسيطة" (Character Ranges) مثل `'a-z'` و `'A-Z'` ، وهي تعتبر الشكل الأبسط والأولي لمطابقة الأنماط (Pattern Matching) المشابهة لمفهوم ال Regex. ولذلك يتم غالباً تدريسه معهم كأداة نصية مكتملة لتغيير الحروف.

2. محرر النصوص VI وشرح السطر الخاص به:

- المثال:

```
:%s/^old/new/g
```

- شرح السطر تفصيلاً:
- `s%` : تعني تطبيق أمر الاستبدال (`Substitute`) على جميع أسطر الملف بالكامل (`%`).
- `/` : الشرطة المائلة تفصل بين أجزاء أمر الاستبدال.
- `old^` : هو جزء ال Regex؛ حيث تعني علامة ال `^` (بداية السطر)، أي أنه يبحث عن كلمة `old` بشرط أن تكون في بداية السطر فقط.
- `new` : الكلمة الجديدة التي سيتم وضعها مكان الكلمة القديمة.
- `g/` : تعني تطبيق التعديل بشكل عام (`Global`) على كل التكرارات داخل السطر الواحد، وليس أول تكرار فقط.

3. تفصيل وشرح أمر ال `sed` وجزء ال Slashes بالتفصيل:

الأمر اللي استخدمناه كان:



Bash

```
sed '/^$/d' file.txt
```

وتفصيل أجزائه كالتالي:

• **خيار -i**: معناه "عدل في الملف مباشرة واكتب النتيجة جواه، من غير ما تطبعها على الشاشة."

○ `sed` : اسم الأمر (Stream Editor) المسؤول عن تعديل النصوص.

○ الشرطة المائلة الأولى والأخيرة (`/ ... /`): تفصل بين النطاق أو التعبير المنتظم (Regex) الذي يبحث عنه ال `sed` داخل الملف.

○ `^` (الواقعة بين الشرطات المائلات): هي تعبير ال Regex؛ حيث تعني `^` بداية السطر، وتعني `$` نهاية السطر، ووجودهما متلاصقين هكذا معناه "السطر الفارغ تماماً" (الذي تبدأ نهايته فور بدايته بدون أي حروف).

○ `d` (في النهاية): هو الاختصار أو الأمر المراد تنفيذه على ما تم العثور عليه، وحرف ال `d` معناه Delete (حذف) السطور التي تطابق الشرط.

○ `file.txt` : اسم الملف المراد تطبيق التعديل عليه.

لو كنا كتبناها من غير أقواس كده `dir_backup.tar$` ، كان الباش هيفتكر إن اسم المتغير بالكامل هو `dir_backup` ، ومش هيلاقى المتغير ده وهيطلع فاضي! الأقواس `{ }` هنا بتحدد حدود اسم المتغير بوضوح وتقول للباش: "هذا هو اسم المتغير (`dir`) فقط والباقي (`backup.tar_`) عبارة عن نص إضافي هتلقاه جنبه".

```
#!/usr/bin/bash
```

```
# Loop through the specified
# sensitive system files/directories
for target in /etc/shadow /etc/passwd /etc
do
    # Check if the target does NOT exist;
    # if true, print a message and skip to the next iteration
    [[ ! -e $target ]] && echo "$target does not exist" && continue

    # Extract the base name and clean it
    # by deleting all slashes completely using tr -d
    filename=$(echo $target | tr -d '/')

    # Create a tar archive
    # of the target file or directory
    tar -cf ${filename}_backup.tar $target

    # Check if the tar archive
    # was created successfully
    [[ $? -eq 0 ]] && echo "Tar archive for $target is done successfully"

    # Transfer the archive to the remote backup server
    # using scp with the exact path provided
    scp -P 22 ${filename}_backup.tar Elhalawany@192.168.1.12:/F:/backup

    # Check if the transfer was successful,
    # then remove the local archive file
    [[ $? -eq 0 ]] && rm -rf ${filename}_backup.tar
done
```


مفهوم Command Substitution (استبدال الأمر في الباش)

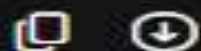
- التعريف: هي تقنية في ال Bash تتيح لك تنفيذ أمر معين واستبداله بالنتائج (Output) الخاص به مباشرة داخل متغير أو سطر أوامر.
- طريقة الكتابة الصحيحة:

```
variable = $(command)
```

(مثال: `filename=$(echo $target | tr -d '/')`)

ماذا لو تم عكس أو إزالة علامة الدولار (\$)؟

- لو كتبت القوسين من غير دولار `variable=(...)` :
 - الباش لن يرى ما بداخل القوسين على أنه "أمر" يتم تنفيذه.
 - بدلاً من ذلك، سيعتبرها مصفوفة (Array)، مما يؤدي إلى تخزين البيانات كعناصر مصفوفة أو إعطاء خطأ (Syntax Error) إذا لم تكن مناسبة للبناء البرمجي.
- الخلاصة: علامة الدولار مع الأقواس (\$) () هي المسؤولة تماماً عن إخباره بأن ما بداخله هو أمر تشغيلي يجب تنفيذه أولاً وأخذ نتيجته لتخزينها.
- آلية عمل الأمر `tr -d '/'` :
 - باستخدام هذا الأمر مع خيار الحذف (d-)، قمنا بإزالة وحذف أي علامة / (Forward Slash) موجودة بالكامل داخل النص (المسار)، وليس فقط الشرطة الأولى.
- الهدف منها:
 - تحويل المسارات المعقدة (مثل `etc/shadow/`) إلى اسم نصي متصل وصافي تماماً (مثل `etcshadow`) لكي نستدمجه بشكل نظيف في تسمية ملف الأرشفة (`etcshadow_backup.tar`) دون وجود أي رموز أو فواصل.



Bash

```
#!/usr/bin/bash
```

```
for dir in /etc /home /tmp
do
```

```
# التحقق من وجود المجلد أولاً # يمكن تشيل علامات التنصيص من على المتغير وتستخدم [] #
```

```
if [ ! -d "$dir" ]; then
    echo "[ERROR]: Directory $dir does not exist. Skipping..."
    continue
fi
```

```
# فقط المجلد والتحقق من نجاح العملية
```

```
echo "Starting backup for: $dir"
tar -cf "${dir}_backup.tar" "$dir" 2>/dev/null
if [ $? -ne 0 ]; then
    echo "[ERROR]: Failed to create tar archive for $dir"
    continue
fi
```

```
# نقل الملف للسيزر البعيد والتحقق من نجاحه قبل الحذف
```

```
scp "${dir}_backup.tar" root@192.168.1.10:/backup
if [ $? -eq 0 ]; then
    echo "[SUCCESS]: Backup for $dir transferred successfully."
    rm -f "${dir}_backup.tar"
else
    echo "[ERROR]: Failed to transfer $dir backup to remote server."
fi
done
```

Summary

In this chapter, you learned:

- How to create and execute simple Bash scripts.
- How to use loops to iterate through a list of items from the command-line and in a shell script.
- How to search for text in log files and configuration files using regular expressions and grep.



RH134

Scheduling Future Tasks



Red Hat | Enterprise Linux

Goal

- Schedule tasks to automatically execute in the future.

Objectives

- Set up a command that runs once at some point in the future.
- Schedule commands to run on a repeating schedule using a user's crontab file.
- Schedule commands to run on a repeating schedule using the system crontab file and directories.
- Enable and disable systemd timers, and configure a timer that manages temporary files.

Scheduling Future Linux Tasks



فوائد وأهمية جدولة المهام في النظام (Benefits of Scheduling Tasks)

تعتبر جدولة المهام أداة أساسية لأي سبستم أدمن، وتلعب دوراً رئيسياً في تحسين أداء العمل من خلال النقاط التالية:

1. أتمتة المهام الروتينية (Automation):

- **المعنى:** تنفيذ المهام المتكررة تلقائياً دون تدخل يدوي مستمر.
- **الأهمية:** توفر الكثير من الوقت والجهد (Save Time and Effort) على مدير النظام، وتضمن تنفيذ العمليات بانتظام.

2. رفع الإنتاجية والتركيز (Productivity & Focus):

- **المعنى:** تحسين إنتاجية مسؤولي النظام (System Administrators).
- **الأهمية:** تتيح للسبستم أدمن التركيز على المهام الحرجة والأكثر أهمية بدلاً من استهلاك وقته في العمليات الروتينية المتكررة.

3. تقليل الأخطاء البشرية (Error Reduction):

- **المعنى:** خفض احتمالية حدوث أخطاء ناتجة عن التنفيذ اليدوي للأوامر (Manual Execution).
- **الأهمية:** الاعتماد على الجدولة الآلية يضمن تنفيذ الأوامر بدقة وبنفس المعايير في كل مرة دون نسيان أي خطوة.

4. تحسين واستغلال موارد النظام (Resource Optimization):

- **المعنى:** جدولة المهام الثقيلة لتعمل في أوقات انخفاض ضغط النظام (Off-Peak Hours).
- **الأهمية:** إذا كان لديك مهمة حرجية تتطلب موارد كبيرة (More Resources)، يمكنك جدولتها لتعمل عندما يكون System Load منخفضاً جداً؛ مما يضمن توفر موارد كافية لتنفيذ المهمة بكفاءة عالية دون التأثير على أداء النظام العام.

Scheduling one-time tasks with at command

- Use the `at` TIMESPEC command to schedule a new job. The **at** command then reads the commands to execute from the stdin channel. While manually entering commands, you can finish your input by pressing Ctrl+D. For more complex commands that are prone to typographical errors, it is often easier to use input redirection from a script file, for example, **at now +5min < myscript**, rather than typing all the commands manually in a terminal window.
- The TIMESPEC argument with the **at** command accepts many powerful combinations, allowing users to describe exactly when a job should run. Typically, they start with a time, for example, **02:00pm**, **15:59**, or even **teatime**, followed by an optional date or number of days in the future.
- The following lists some examples of combinations that can be used.
 - **now +5min**
 - **teatime tomorrow (teatime is 16:00)**
 - **noon +4 days**
 - **5pm august 3 2021**

جدولة المهام لمرة واحدة باستخدام أمر `at` (Scheduling One-Time Tasks)

يُستخدم أمر `at` لتنفيذ المهام لمرة واحدة في وقت مستقبلي محدد، وينقسم شرحه إلى الأقسام التالية:

1. طريقة عمل الأمر والإدخال (How it Works):

- استخدام الأمر: يُكتب بالشكل `at TIMESPEC` لجدولة مهمة جديدة.
- مدخلات الأوامر (`stdin`): يقرأ أمر `at` الأوامر المراد تنفيذها من قناة الإدخال القياسي `stdin`.
- إنهاء الإدخال اليدوي: أثناء كتابة الأوامر يدوياً في التيرمينال، يمكنك إنهاء وحفظ الإدخال بالضغط على اختصار لوحة المفاتيح `Ctrl + D`.
- إعادة التوجيه (Input Redirection): للأوامر المعقدة المعرضة لأخطاء الكتابة، يُفضل استخدام ملف سكريبت بدلاً من الكتابة اليدوية، مثل: `at now +5min < myscript` لتجنب الأخطاء الإملائية (Typographical Errors).

2. صيغة تحديد الوقت : TIMESPEC

- المرونة: يقبل مُعامل `TIMESPEC` تركيبات قوية ومتعددة لتحديد وقت تشغيل المهمة بدقة.
- النمط العام: يبدأ عادةً بتحديد الوقت (مثل: `02:00pm` أو `15:59` أو `teatime`)، متبوعاً الاختياريّاً بالتاريخ أو عدد الأيام في المستقبل.

3. أمثلة عملية على صيغ الوقت المستخدمة:

- `now +5min` : لتنفيذ المهمة بعد 5 دقائق من الآن.
- `teatime tomorrow` : لتنفيذ المهمة غداً وقت الشاي (حيث أن `teatime` تعني عادة الساعة `16:00`).
- `noon +4 days` : لتنفيذ المهمة عند الظهر بعد 4 أيام.
- `5pm august 3 2021` : لتنفيذ المهمة في تمام الساعة الخامسة مساءً في تاريخ محدد.

Inspecting and managing deferred user jobs

فحص وإدارة المهام المؤجلة

- To get an overview of the pending jobs for the current user, use the command **atq** or the **at -l** commands. لعرض نظرة عامة على جميع المهام المعلقة (pending jobs) الخاصة بالمستخدم الحالي.

```
[user@host ~]$ atq
```

```
❶28 ❷Mon Feb  2 05:13:00 2015 ❸a ❹user  
29  Mon Feb  3 16:00:00 2014 h user  
27  Tue Feb  4 12:00:00 2014 a user
```

In the preceding output, every line represents a different job scheduled to run in the future.

- ❶ The unique job number for this job.
- ❷ The execution date and time for the scheduled job.
- ❸ Indicates that the job is scheduled with the default queue a. Different jobs may be scheduled with different queues.
- ❹ The owner of the job (and the user that the job will run as).

💡 ملحوظة لمدير النظام (System Administrator Notes):

- ماذا تعني الحروف المختلفة (مثل `a` و `h`) في عمود الطابور؟
 - الحرف الظاهر بجانب المهمة يمثل حرف الطابور (Queue Letter) الذي تنتمي إليه المهمة لتنظيمها وتصنيفها داخل النظام.
 - الحرف الافتراضي لأي مهمة جديدة تنشئها هو `a`.
- تأكيد مهم (الأولوية وسرعة المعالج):
 - تنبيه: هذه الحروف لا تعطي أبداً أي أولوية في التنفيذ أو سرعة معالج أعلى (CPU Priority / Processing Speed). النظام لا يعامل حرفاً بشكل أسرع أو أقوى من حرف آخر.
 - الهدف الحقيقي: هو لتنظيم إداري وتصنيف فقط؛ بحيث تكون عارف ومميزت أن المهمة دي مثلاً خاصة بالباك اب (Backup) فواحدة حرف `h` ، ومهمة ثانية تابعة لحاجة مختلفة واحدة حرف `b` ، وهكذا. عشان لما تعرض المهام بـ `atq` تقدر تفرزها وتعرف تصنيفاتها بسهولة كسيستم أدمن.
- كيف تحدد وتخصص طابوراً لمهمتك؟
 - بصفتك مدير نظام، يمكنك توجيه مهمة معينة لطابور محدد عند إنشائها باستخدام خيار أمر `at` مع مفتاح `-q` متبوعاً بحرف الطابور (من `a` إلى `z`).
 - مثال عملي: لو عندك مهمة نسخ احتياطي وتريد تميزها بالحرف `h` ، يمكنك كتابة الأمر هكذا:



Bash

```
at -q h 02:00am tomorrow < backup_script.sh
```

- النتيجة: عندما تقوم بعرض المهام لاحقاً باستخدام `atq` ، سيظهر بجانبها الحرف `h` لتدل على تصنيفها الخاص بالباك اب.

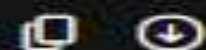
Create at task

- **at +time** (to make a schedule for some commands to run in this time then enter.
- Time like now,1hour,6AM,tomorrow,4PM+3days , 10AM Jul 31, teatime ---> 4 pm). Then click enter
- **at>** write the commands you want to execute in this time each command in one line then CTRL +d ---> to finish.
- **atq** -----> show you all schedules created.
- **atrm + job** -----> to delete the schedule. With schedule Id
- **Time can be:**
 - HH:MM 23:15
 - Midnight 12:00 am
 - Noon 12:00 pm

التطبيق العملي لأمر `at` وخطواته بالترتيب:

1. جدولة مهمة في وقت الظهيرة (`noon`):

○ الأمر:



Bash

```
at noon
echo "Hello" >>/tmp/hello.txt
```

○ الإنهاء: الضغط على `Ctrl + D` لإنهاء الإدخال وحفظ المهمة.

2. جدولة مهمة بعد 3 أيام من الظهيرة (`noon +3days`):

○ الأمر:



Bash

```
at noon +3days
mkdir /tmp/backup_system
tar -cvf /tmp/backup_system/etc.tar /etc
```

○ الإنهاء: الضغط على `Ctrl + D`.

3. التحقق من التاريخ الحالي وعرض المهام المجدولة:

○ الأمر: `date`

○ الأمر: `atq` (أو `at -l`) لعرض قائمة المهام المعلقة الحالية.

4. حذف وإلغاء المهام:

- الأمر لحذف مهمة برقمها: `atrm 21`
- عرض القائمة مجدداً للتأكد: `atq`
- طريقة أخرى لحذف مهمة: `at -r 22` ثم التحقق بـ `at -l`.

5. جدولة مهمة في منتصف الليل (`midnight`):

- الأمر:



Bash

```
at midnight  
mkdir /tmp/system
```

- الإنهاء: الضغط على `Ctrl + D`.

6. كتابة الأمر بصيغة خاطئة:

- الأمر:



Bash

```
at teamtime
```

7. جدول مهمة وقت الشاي (teatime) :

○ الأمر:

Bash

```
at teatime
touch /tmp/file1
rm -rf /tmp/file1
```

○ الإنهاء: الضغط على `Ctrl + D`.

8. إنشاء مهمة بسطر واحد (Inline) :

○ الأمر:

Bash

```
echo "date >> /home/karim/myjob.txt" | at now
```

○ التحقق من الملف: `cat /home/karim/myjob.txt`

9. إعادة توجيه ملف غير موجود:

○ الأمر:

Bash

```
at now +5min </home/karim/script.sh
```

○ الشرح: يُستخدم هذا الأمر لعمل إعادة توجيه (Redirection) لملف سكريبت خارجي يحتوي على الأوامر بدلاً من كتابتها يدوياً.

10. مراجعة ملفات مهام الـ `at` مباشرة من مسار التخزين المؤقت:

○ الأمر:



Bash

```
at teatime
echo "It's teatime">> /home/karim/tea.txt
```

○ التحقق من مسار التخزين وملفات الـ `Spool`:



Bash

```
watch atq
cd /var/spool/at/
ls
cat <job_filename>
```

المراقبة الحية: (Real-time Monitoring) في المثال بتاعك (`watch atq`) ، هو بيخلي السيستم يعرض لك قائمة المهام المعلقة ويحدثها كل ثانيتين، فلو فيه مهمة وقتها جه واتفذت واختفت من اللىسة، هتشفو بعينك وهي بتختفي فوراً من غير ما تقعد تكتب `atq` تاني .

11. عرض تفاصيل ومحتوى مهمة معينة:

○ الأمر:



Bash

```
at -c 26
```

○ الشرح: تستخدمه في حالة أنك تريد استعراض تفاصيل السكريبت أو الجواب المعينة والفايل تيم الخاص بها المرتبط برقم المهمة (`Job ID`).

At Job Important Files

- **/etc/at.allow** (the file which has the users who can use at commands).
- **/etc/at.deny** (the file which has the users who denied from using at commands).
- **/var/spool/at/** (Holds the jobs script files).

Notes

- If the both files not exists that is mean all users are denied.
- The default installation create **at.deny** and make as default all users allowed to use at commands so take care.

ملفات التحكم وصلاحيات استخدام `at` (At Job Important Files)

يتحكم النظام في صلاحيات المستخدمين الذين يمكنهم استخدام أوامر الـ `at` من خلال ملفين أساسيين ومسار تخزين:

1. الملفات الهامة ومساراتها:

- `etc/at.allow/` : الملف الذي يحدد المستخدمين المسموح لهم حصرياً باستخدام أوامر الـ `at`.
- `etc/at.deny/` : الملف الذي يحدد المستخدمين الممنوعين من استخدام أوامر الـ `at`.
- `/var/spool/at/` : المسار الذي يحفظ ملفات السكريبت الخاصة بالمهام (Holds the jobs script files).

2. قواعد الصلاحيات وآلية عمل النظام (Access Control Rules):

- **وجود ملف `at.allow` فقط:** إذا أنشأ مدير النظام هذا الملف ووضع بدايته (مثلاً اسم المستخدم `ahmed`)، فإن النظام يسمح لهذا المستخدم فقط ويقوم بحظر (`deny`) باقي مستخدمي السيستم بالكامل.
- **إذا لم يوجد ملف `at.allow`:** ينتقل النظام لفحص ملف `at.deny` ؛ فإذا وُجد اسم المستخدم بداخله يتم حظره، وإذا كان الملف فارغاً أو غير ممنوع، يُسمح لجميع المستخدمين العاديين (`regular users`) باستخدام أوامر الـ `at`.
- **الحالة الخاصة (الملفان موجودان وفارغان أو ممسوحان):** إذا لم يكن أي من الملفين موجوداً تماماً، أو كان كلاهما موجوداً وفارغاً، فإن السلوك الافتراضي (`Default Behavior`) للنظام هو حظر جميع المستخدمين (`All users are denied`).

3. ملاحظات هامة لمدير النظام (Important Notes):

- التثبيت الافتراضي للنظام يقوم بإنشاء ملف `at.deny` فقط ويتركه فارغاً، مما يعطي صلاحية السماح لجميع المستخدمين افتراضياً. لذا يجب الحذر وتوخي الحيلة (The default installation creates `at.deny` and makes `as default all users allowed`)

التطبيق العملي لاختبار صلاحيات أمر الـ `at` (سيناريو الروت ويوزر أحمد):

1. إضافة يوزر أحمد إلى ملف الحظر (`at.deny`):

- من شاشة الـ `root` ، قام بكتابة اسم أحمد داخل ملف الحظر لمنع من استخدام أوامر الـ `at` :



Bash

```
echo ahmed > /etc/at.deny
```

2. اختبار الصلاحية من تيرمينال يوزر أحمد:

- بالانتقال لتيرمينال يوزر أحمد (`su - ahmed`) ومحاولة جدولته لمهمة `at teatime` ، تم قبول الجواب مبدئياً (`job 28`) ، ولكن عند محاولة استعراض المهام باستخدام `atq` ظهرت رسالة الخطأ:



Plaintext

```
You do not have permission to use atq.
```

- 💡 **ملحوظة وتوضيح تقني:** أحياناً قد يتم قبول إنشاء المهمة لأول وهلة بسبب تأثير النظام بحالة الكاش المؤقتة (`Cache`) أو التأخر البسيط في تحديث تطبيق صلاحيات الملف لحظياً، ولكن السياسة الأساسية تمنع المستخدم المحظور من الاستخدام الكامل، ولو قام بالتجربة بعد لحظات أو تم تحديث الجلسة فلن يتمكن من ذلك.

4. التحقق من نجاح يوزر أحمد بعد إزالة الحظر:

- بالعودة لتيرمينال يوزر أحمد وتنفيذ `atq` مجدداً، ظهرت المهمة المعلقة بنجاح (`atrm 28 Tue Jul 4 16:00:00 2023 a ahmed 28`) وتمكن من حذفها باستخدام الأمر `atrm 28` بعد إزالة الحظر عنه.

5. التعامل مع وجود الملفين معاً وفراغهما:

- من شاشة الـ `root` ، قام بتفريغ ملف الحظر وتفريغ أو إنشاء ملف السماح `at.allow` وجعله فارغاً تماماً:



Bash

```
> /etc/at.deny  
> /etc/at.allow
```

6. ظهور مشكلة عدم الصلاحية مجدداً:

- عند محاولة يوزر أحمد تنفيذ `atq` بعد أن أصبح الملفان موجودين وفارغين في نفس الوقت، ظهرت رسالة الحظر مرة أخرى (`You do not have permission to use atq`)، وهو ما يثبت القاعدة بأن وجود الملفين معاً وفراغهما يؤدي إلى حظر جميع المستخدمين افتراضياً.

7. منع الصلاحية الحصرية ليوزر أحمد عبر `at.allow`:

- من شاشة الـ `root` ، قام بكتابة اسم أحمد داخل ملف السماح المخصص:



Bash

```
echo ahmed > /etc/at.allow
```

- وبذلك يقتصر استخدام أوامر الـ `at` على يوزر أحمد وحده ويتم منع باقي مستخدمي النظام.

Describing recurring user jobs

- Jobs scheduled to run repeatedly are called recurring jobs. Red Hat Enterprise Linux systems ship with the **crond** daemon, provided by the **crontie** package, enabled and started by default specifically for recurring jobs. The **crond** daemon reads multiple configuration files: one per user (edited with the **crontab** command), and a set of system-wide files. These configuration files give users and administrators fine-grained control over when their recurring jobs should be executed.
- Fields in the **crontab** file appear in the following order:
 - Minutes.
 - Hours.
 - Day of month.
 - Month.
 - Day of week.
 - Command.

وصف المهام المتكررة للمستخدم (Describing recurring user jobs)

- مقدمة عن خدمة الكرون (Cron Service):
- خدمة الـ `cron` أو كما تُعرف بـ (`crond daemon`) القادمة ضمن حزمة `package cronie` هي المسؤولة عن تشغيل المهام بشكل دوري ومتكرر (`periodically & repetitively`) لجدولة أي مهام متكررة أو متجددة (`replicated tasks`).
- تأتي هذه الخدمة مثبتة ومفعلة بشكل افتراضي في أنظمة `Red Hat Enterprise Linux` (`enabled and started by default`).
- ينزل مع خدمة الكرون مجموعة من أدوات الأوامر، أهمها أمر `crontab` الذي يُستخدم لجدولة وإدارة هذه المهام المتكررة.
- الفرق بين أمر `at` وخدمة `cron` :
- أمر الـ `at` : مخصص لـ مرة واحدة فقط (`One-time execution`)؛ بمعنى أنه يجدول مهمة لتنفيذ وقتاً طبعاً في المستقبل، ومجرد ما الموعد ييجي وتنفيذ المهمة بتت حذف تماماً لوحدها.
- خدمة الـ `cron` : مخصصة للمهام المتكررة (`Recurring / Repetitive tasks`) والدورية بشكل مستمر.

كيف تعمل خدمة crond وملفات المستخدمين؟

- يقوم الـ `crond daemon` بقراءة عدة ملفات ضبط وإعداد (`configuration files`)؛ منها ملف مخصص لكل مستخدم يتم تعديله والتعامل معه من خلال أمر `crontab` ، بالإضافة إلى مجموعة من الملفات الخاصة بالسيستم ككل (`system-wide files`).
- تمنح هذه الملفات المستخدمين ومديري النظام تحقماً دقيقاً وعميقاً في توقيت تنفيذ مهامهم المتكررة.
- ملف كل يوزر وتوضيح فكرة الـ 50 مهمة: كل يوزر عادي في النظام سيكون لديه ملف `crontab` واحد خاص بيه لوحده يقدر يظبط فيه مهامه. وإذا كان لديك مثلاً 50 مهمة وتريد لـ 10 منها أن تتكرر كلها الساعة 12، فلن تحتاج إلى عدة ملفات؛ بل هو ملف واحد فقط لكل يوزر، وتقوم بكتابة الـ 10 مهام (أو الـ 50 مهمة) داخل نفس الملف في أسطر منفصلة (كل سطر يمثل مهمة وتوقيتها الخاص)، وسيقوم النظام بتنفيذها كلها في مواعيدها المحددة.

صلاحيات وتحكم الروت (Administrator Control):

- بصفتك `root` أو `Administrator` ، لديك الصلاحية الكاملة للاطلاع على ملفات الـ `crontab` الخاصة بجميع المستخدمين على السيرفر وتعديلها أو مراقبتها.
- كما يمكن للروت التحكم في صلاحيات الوصول للخدمة باستخدام ملفات الحظر والسماح الخاصة بالكرون مثل (`cron.allow` و `cron.deny`) للسماح أو منع المستخدمين من استخدامها.

عند استخدام أمر **crontab -e** بنفسك، يقوم النظام بفتح الملف الخاص بك مباشرة والمخزن أوتوماتيكياً في مسار التخزين الداخلي للكروونات وباسمك، وبمجرد كتابة السطر الخاص بالمهام وحفظ الملف، يتم إنشاؤه وتحديثه تلقائياً لتبدأ المهام في العمل في مواعيدها .

Create crontab

```
# For details see man 4 crontabs

# Example of job definition:
# ----- minute (0 - 59)
# | ----- hour (0 - 23)
# | | ----- day of month (1 - 31)
# | | | ----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | ----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | | option
# * * * * * user-name command to be executed
```

- **crontab -e** (to create a cron , you should enter the full path of the command which you want to execute it) which + command (to get the full path of the command).
- ***/3** (in the minutes field means every 3 minutes , in the hours field --> means every 3 hours ...etc).

- root ➤ **crontab -e -u +user** (to edit in cron of specific user). ➤ **crontab -l** (to list cron scheduled).
- **crontab -l -u +user** (to list the crons of specific user). ➤ **crontab -r** (to remove your cron).

Scheduling user jobs

- Normal users can use the **crontab** command to manage their jobs. This command can be called in four different ways:

Crontab Examples

COMMAND	INTENDED USE
<code>crontab -l</code>	List the jobs for the current user.
<code>crontab -r</code>	Remove all jobs for the current user.
<code>crontab -e</code>	Edit jobs for the current user.
<code>crontab filename</code>	Remove all jobs, and replace with the jobs read from <i>filename</i> . If no file is specified, stdin is used.

crontab filename النقطة الغامضة: تقوم هذه الطريقة بحذف جميع المهام القديمة الموجودة في جدولك الحالي بالكامل، واستبدالها بالمهام التي سيتم قراءتها من ملف خارجي (filename) تختاره أنت. وإذا لم يتم تحديد اسم ملف، سيتم قراءة المدخلات مباشرة من الشاشة. (stdin).

Example recurring user jobs

- The following job executes the command `/usr/local/bin/yearly_backup` at exactly 9 a.m. on February 2nd, every year.

```
0 9 2 2 * /usr/local/bin/yearly_backup
```

- The following job sends an email containing the word **Chime** to the owner of this job, every five minutes between 9 a.m. and 5 p.m., on every Friday in July.

```
*/5 9-17 * Jul 5 echo "Chime"
```

- أي مخرجات (Output) أو رسائل خطأ تطلع من تنفيذ أي أمر كرون، السيستم بيجمعها أوتوماتيكياً ويبعتها في إيميل محلي (Local Email) لمالك المهمة (اليوزر اللي شغلها) على نفس السيرفر، عشان يقدر يدخل يشيك عليها ويعرف هل الجوب اتنفذت صح ولا حصلت مشكلة.
- The following job runs the command `/usr/local/bin/daily_report` every weekday at two minutes before midnight.

```
58 23 * * 1-5 /usr/local/bin/daily_report
```

مثال عملي لكتابة ومتابعة مهام الكرون (Cron Job Example & Monitoring)

- الخطوة الأولى: فتح ملف الكرون وكتابة المهمة

- افتح ملف الكرون الخاص بك باستخدام الأمر:

```
crontab -e
```

- أضف هذا السطر لتنفيذ الأمر مباشرة في ملف الكرون:

```
"echo "Test Script Executed at $(date) * * * * *
```

- ملحوظة هامة (لو لديك سكريبت جاهز):

- لو كنت تريد تشغيل ملف سكريبت (sh.) بدلاً من أمر مباشر، يمكنك كتابة المسار الكامل له في نفس السطر هكذا:

```
path/to/script.sh/ * * * * *
```

- الخطوة الثانية: كيفية المتابعة (Log vs Mail)

- 1. المتابعة الحية باستخدام الـ **Log** (متى تستخدمها؟):

- تستخدمها كـ System Admin لو أنت واقف قدام الـ Terminal وعايز تتأكد بلحظتها أن المهمة اتنفذت في وقتها الدقيق وسجلت بنجاح في الخلفية من غير تأخير.

- الأمر: `sudo tail -f /var/log/cron`

- 2. متابعة النتائج التفصيلية باستخدام الـ **Mail** (متى تستخدمها؟):

- تستخدمها لو أنت عارف إن المهمة اتنفذت في وقت سابق (مثلاً الساعة 5:00 وأنت لم تكن متواجداً أو مش مشغول)، وتريد لاحقاً في أي وقت (بكرة أو بعده) الدخل للتحقق وقراءة التفاصيل والمخرجات الكاملة لمعرفة ما نتج عن تنفيذ هذا الأمر بالتحديد.

- الأمر: `mail` (ثم تختار رقم الرسالة لعرض المحتوى وتضغط `q` للخروج).

Example recurring user jobs

- The following job executes the **mutt** command to send the mail message **Checking in** to the recipient **boss@example.com** on every workday (Monday to Friday), at 9 a.m.

```
0 9 * * 1-5 mutt -s "Checking in" boss@example.com % Hi there boss, just checking in.
```

تنفذ هذه المهمة أمر الـ **mutt** لإرسال رسالة بريد إلكتروني تحمل العنوان **Checking in** إلى المستلم **boss@example.com** في كل أيام العمل (من الإثنين إلى الجمعة)، وذلك تمام الساعة 9 صباحاً.

Crontab examples

أي كولوم (عمود) مش عايز تحدد فيه قيمة معينة، أو الدكتور/المحاضر مجابش سيرته في السؤال، تحط مكانه نجمة (*) فوراً من غير ما تفكر !

0	8	*	*	*	Once_a_day
0,30	9	*	*	*	Twice_a_day
0,30	8-18	*	*	*	Twenty_Two_times_a_day
*/5	*	*	*	*	Every_five_minutes
12	13	1	*	*	Once_a_month
49	23	16	9	*	Once_a_year
0	15	*	*	1	Every_monday
32	14	1	1	1	??? (caveat!) Caveat

مرتين يومياً (الساعة 9:00 والساعة 9:30).
هذا يعتبر شرط معقد

هنا لما حددت يوم 1 (يوم الشهر) وشهر 1 (يناير) ويوم 1 (الاثنين)، كأنك بتقول للسيستم: "نفذ المهمة يوم 1 يناير، وبشرط أن يوافق هذا اليوم يوم إثنين".

ملحوظة هامة لسيستم أدمن محترف:
احرص دائماً على عمل نسخة احتياطية (Backup) لجدول الـ Crontab الخاص بك في ملف خارجي، حتى تتمكن من استعادتها فوراً (restore) في حال حدوث أي خطأ أو حذف مقصود.

ملحوظة حول مسار تخزين الملفات:
جداول الـ Crontab الخاصة بكافة المستخدمين يتم تخزينها في المسار الأساسي `var/spool/cron/` ، وصلاحيه الدخول ومشاهدة محتويات هذا المجلد مقتصرة على المستخدم `root` فقط للحفاظ على الأمان.

Block 1: Root Crontab Management, Examples, and Backup (English Comments):

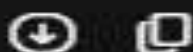
Bash

```
# 1. Create and edit root crontab, followed by the three crontab examples
crontab -e
* * * * * echo "Hello World!" >> /tmp/hello.txt
0 9 * * * /root/Desktop/backup.sh
0 8 1 1 1 yearly_backup

# 2. List root crontab tasks, check calendar, backup, cat, restore, and delete
crontab -l
cal 1 2024
cal 1 2025
crontab -l > crontab_backup
cat crontab_backup
crontab crontab_backup # Restore crontab from backup file
crontab -r
crontab -l
```

Block 2: Other Users Management and Spool Permissions (English Comments):

Bash



```
# 1. Manage other user's crontab from root (user: karim)
crontab -l -u karim
crontab -e -u karim
crontab -r -u karim
crontab -l -u karim # Verify no crontab for karim

# 2. Switch to user karim and test creating/listing/deleting their cron
su - karim
crontab -e
*/5 * * * * echo "Hello from karim" >> /tmp/karim.txt
crontab -l
crontab -r
crontab -l # no crontab for karim

# 3. Navigate to spool directory, check files, and verify permissions
cd /var/spool/cron/
ls
cat karim
cat root
pwd
# Show that permissions are restricted to root only (drwx-----)
ls -ld /var/spool/cron/ # and other users have no access
```


Cron Job Important Files

- **/etc/cron.d** (This is the path to put (your) cron files created).
- **/etc/cron.deny** (the file to put who users you want to deny them from creating cron).
- **/etc/cron.allow** (to put the users who allowed to create cron).
- **/var/spool/cron** (it contains all the crons of all users, so you can access any file you want).
- **/etc/crontab** (to see the syntax for writing a cron task).
- **/etc/anacrontab** (its a file contain all scripts runs in specific time , if you cat this file you will see all scripts which will run in future and time ...etc).
- **/etc/cron.daily** (list scripts which will be executed daily).
- **/etc/cron.monthly** (list scripts which will be executed every month).

ملفات نظام ال Cron الهامة (Cron Job Important Files)

- o `etc/cron.d/` : المسار المخصص لوضع ملفات الكرون المخصصة (`customized cron files`) ومميزته الأساسية أن تحديث الحزم العادية (`package updates`) لا يتسبب في عمل `overwrite` أو مسح للملفات الموجودة بداخله، على عكس ال `crontab` العادية.
- o `etc/cron.deny/` : الملف الذي يُستخدم لتحديد واستثناء المستخدمين الممنوعين من إنشاء مهام كرون (`deny them from creating cron`).
- o `etc/cron.allow/` : الملف الذي يحدد المستخدمين المسموح لهم بإنشاء الكرون (`allowed users to create cron`), وغالباً بالوضع الافتراضي غير موجود أو مخصص للتحكم الحصري في النظام.
- o `var/spool/cron/` : المجلد الذي يحتوي على جميع ملفات ال `crontab` الخاصة بكافة مستخدمي النظام، بحيث يمكنك الوصول إلى أي ملف يوزر تحتاجه (بصلاحيات الروت فقط).
- o `etc/crontab/` : ملف النظام الرئيسي الذي يوضح ال `syntax` والقورمات الصحيحة لكتابة مهام الكرون وتنسيقها.
- o `etc/anacrontab/` : ملف إدارة المهام المؤجلة؛ ومميزته مقارنة بالكرون العادي أنه إذا جاء وقت المهمة وكان الجهاز مغلقاً (`Power off`) أو تم إعادة تشغيله (`Reboot`), فإن ال Cron العادي يتجاهلها (`Ignore`), بينما ال Anacron ينتظر أول ما الجهاز يفتح (`Power on`) وينفذ السكريبت فوراً.
- o `etc/cron.daily/` : مجلد لتشغيل السكريبتات بشكل يومي (`executed daily`), وبدون الحاجة لكتابة النجوم لتحديد الوقت، فالنظام يتكفل بتشغيلها دورياً عبر ال `anacron`.
- o `etc/cron.monthly/` : مجلد لتشغيل السكريبتات بشكل شهري منتظم (`executed every month`).

ملحوظة هامة: الملفات مثل `daily` و `monthly` يكون متحدد لها وقت محدد بواسطة ال `anacron` (غالباً بعد عمل `Boot` للجهاز بوقت محدد معين).



Bash

```
[root@client ~]# cat /etc/cron.deny
[root@client ~]# echo "karim" > /etc/cron.deny
[karim@client ~]$ crontab -l
[root@client ~]# echo "karim" > /etc/cron.allow
[karim@client ~]$ crontab -l
[root@client ~]# rm -rf /etc/cron.allow /etc/cron.deny
[karim@client ~]$ crontab -l
[root@client ~]# > /etc/cron.deny
[karim@client ~]$ crontab -l
```

تتبع ومعاينة مسار ال Cron.d وملف ال 0hourly:

○ الخطوة الأولى (الدخول للمسار واستعراض المحتويات):

```
ls | cd /etc/cron.d
```

○ **الملحوظة الفنية:** نلاحظ هنا وجود ملف افتراضي أساسي اسمه `0hourly` ، وهو مخصص لجدولة وإدارة المهام بالساعة داخل النظام دون أن تتأثر بـ Overwrite عند تحديث الحزم.

○ الخطوة الثانية (عرض محتوى الملف وتفصيل ال Syntax):

```
cat @hourly
```

○ الشرح التفصيلي للسطر الموجود: `root run-parts /etc/cron.hourly * * * * @1`

○ أول جزئية (`* * * * @1`): تعني أن الأمر سيعمل في الدقيقة الأولى من كل ساعة بانتظام.

○ جزئية (`run-parts`): هو أمر نظامي وظيفته المرور على مجلد بالكامل وتنفيذ كافة السكريبتات والملفات الموجودة بداخله دفعة واحدة.

○ النتيجة: أي سكريبت يتم وضعه داخل مجلد `etc/cron.hourly/` سيتم تنفيذه تلقائياً كل ساعة وبدون الحاجة لكتابة أي نجوم أو توقيتات داخله.

○ الخطوة الثالثة (دخول مجلد hourly وفحص السكريبتات داخل ال bash):



Bash

```
cd /etc/cron.hourly
ls -l
less @anacron
```

○ الملاحظة الاحترازية لسيستم أدمن: ملف ال `@anacron` الذي يتم تشغيله كل ساعة لا ينفذ المهام بنفسه، بل وظيفته الأساسية هي عمل فحص دوري (Check) للتأكد مما إذا كانت مهام ال `anacron` اليومية قد أُجريت اليوم أم لا (`Check whether @anacron was run today already`). وذلك لضمان التعامل مع حالات انقطاع التيار الكهربائي أو إغلاق النظام ومعالجة المهام الفائتة.

○ الأمر المستخدم:

`cat /etc/anacrontab`

○ ملحوظات وتوضيحات الأعمدة:

○ توضيح الـ `RANDOM_DELAY` والـ `START_HOURS_RANGE`: إضافة تأخير عشوائي يصل إلى 45 دقيقة فوق التأخير الأساسي للمهام، بشرط ألا يتم التنفيذ إلا داخل النطاق الزمني المسموح به فقط (من الساعة 3 صباحاً وحتى 10 مساءً `START_HOURS_RANGE=3-22`).

○ العمود الأول (`period in days`): يحدد كل كم يوم يتم تكرار المهمة (مثلاً 1 يعني يومياً، 7 يعني أسبوعياً، `monthly@` يعني شهرياً).

○ العمود الثاني (`delay in minutes`): الوقت الأساسي بالدقائق الذي ينتظره النظام بعد إقلاع الجهاز أو بعد مرور الفترة المحددة قبل تنفيذ الأمر.

○ العمود الثالث (`job-identifier`): اسم تعريفي أو وصف للمهمة (مثل `cron.daily` أو `cron.weekly`) لتسهيل متابعتها في السجلات (`logs`).

○ العمود الرابع (`command`): الأمر الفعلي لتنفيذ السكريبتات، وغالباً يكون متبوعاً بالمسار (مثل `nice run-parts /etc/cron.daily`).

ملحوظة هامة (توزيع اللود): النتيجة النهائية للتأخير هي مجموع (الـ 5 دقائق الأساسية + رقم عشوائي من 0 إلى 45 دقيقة يختاره النظام)، وذلك لمنع الضغط المفاجئ وتوزيع اللود على السيرفر في حال فتح أجهزة متعددة في نفس الوقت بعد انقطاع التيار الكهربائي.

Managing temporary files

- Some applications (and users) use the **/tmp** directory to hold temporary data, Red Hat Enterprise Linux 7 and later include a new tool called **systemd-tmpfiles**, which provides a structured and configurable method to manage temporary directories and files.
- When systemd starts a system, one of the first service units launched is **systemd-tmpfiles** setup. This service runs the command **systemd-tmpfiles --create --remove**. This command reads configuration files from **/usr/lib/tmpfiles.d/*.conf**, **/run/tmpfiles.d/*.conf**, and **/etc/tmpfiles.d/*.conf**. Any files and directories marked for deletion in those configuration files is removed, and any files and directories marked for creation (or permission fixes) will be created with the correct permissions if necessary.
- To ensure that long-running systems do not fill up their disks with stale data, a **systemd** timer unit called **systemd-tmpfiles-clean.timer** triggers **systemd-tmpfiles-clean** service on a regular interval, which executes the **systemd-tmpfiles --clean** command.

إدارة الملفات المؤقتة و وحدات المؤقتات (Managing temporary files & Timer Units):

- الفكرة والنشأة (من ربهات 7 فما بالك):

- مع تطور الأنظمة، ظهرت أدوات حديثة لإدارة النظام تسمى `systemd-tmpfiles` لتوفير طريقة منظمة وقابلة للضبط (`structured and configurable method`) لإدارة المجلدات والملفات المؤقتة التي تملأ مسار الـ `tmp/` عادة من قبل التطبيقات والمستخدمين.

- آلية العمل عند بدء التشغيل (Startup Workflow):

- عند إقلاع النظام، يتم إطلاق خدمة أساسية اسمها `systemd-tmpfiles setup` ، والتي تنفذ الأمر الشهير:

```
systemd-tmpfiles --create --remove
```

- هذا الأمر يقوم بقراءة ملفات الإعدادات من المسارات الثلاثة الشهيرة (`usr/lib/tmpfiles.d/*.conf/` ، `run/tmpfiles.d/*.conf/` ، `etc/tmpfiles.d/*.conf/`)، ليقوم بحذف الملفات المعلمة للحذف، وإنشاء أو ضبط صلاحيات الملفات المطلوبة.

- الـ Timer Unit الحديثة (بديل الـ Cron المتقدم):

- لضمان عدم امتلاء أقراص الأنظمة التي تعمل لفترات طويلة بالبيانات القديمة والمتراكمة (`stale data`)، يوجد `systemd timer unit` يسمى `systemd-tmpfiles-clean.timer`

- هذا التايمر يقوم بعمل `triggers` أو إطلاق لخدمة تسمى `systemd-tmpfiles-clean.service` على فترات منتظمة، والتي بدورها تنفذ الأمر `systemd-tmpfiles --clean` لتنظيف الملفات المؤقتة بانتظام.

- **الكرون القديم (Cron)**: كان يعتمد على ملفات زي `crontab` وتحدد تحسب نجوم ودقائق وساعات، وكان لو السيرفر اتقفل في الوقت اللي المفروض المهمة تشتغل فيه، ممكن تتفوت (إلا لو استخدمنا `Anacron`).
- **التكنولوجيا الحديثة (Timer Units) في Systemd**: ربهات (ابتداءً من الإصدار 7) قالت لك: بدل ما نستخدم على الكرون القديم في كل حاجة، هنعمل نظام جوه مدير النظام نفسه (`systemd`) اسمه `Timer`. ده عبارة عن "منبه" رقمي دقيق جداً مرتبط بخدمة معينة (`Service`).

مثال يبسطها لك تماماً:

تخيل عندك مجلد المؤقتات `tmp/` ده عامل زي "سلة مهملات" اللابتوب أو السيرفر، كل البرامج بترمي فيه زبالتها وفضلاتها المؤقتة. لو سبيناه كده، المساحة هتتملي على الآخر والهادر هيتفسي!

عشان نحل المشكلة دي:

1. النظام محتاج كل فترة معينة يشغل عملية تنظيف (`systemd-tmpfiles --clean`).
2. بدل ما نخط الأمر ده في ملف `cron.daily` العادي، ربهات عملت `Timer Unit` مخصص اسمه `systemd-tmpfiles-clean.timer`.
3. ال `Timer` ده شغال زي المنبه بالضبط؛ أول ما بيحي معنده (مثلاً كل ساعة أو كل يوم)، بيروح يصحدي الخدمة المسؤولة (`systemd-tmpfiles-clean.service`) ويقول لها: "قومي نضفي المجلد دلوقتي!".

باختصار: ال `Timer Unit` هو "الجيل الحديث والمطور" من فكرة الكرون، بس شغال بذكاء وسجلات وإمكانيات مدير النظام (`systemd`) عشان ينفذ نفس المهام الدورية بس بشكل أحدث وأدق.

استعراض وتشغيل التايمر يونتس (Systemd Timers):

○ الأمر المستخدم:

```
systemctl list-units --type=timer
```

○ الملحوظات والشرح الاحترافي:

○ **الفكرة العامة:** بدلاً من استخدام ملفات الكرون التقليدية (`traditional cron files`)، تستخدم أنظمة ريدهات الحديثة وحدات الـ Systemd Timers كمنبهات رقمية دقيقة مرتبطة بمدير النظام لتشغيل المهام في أوقات محددة.

○ أبرز التايمرات والخدمات الظاهرة:

○ `systemd-tmpfiles-clean.timer` : تقوم بتنظيف الملفات المؤقتة دورياً لحماية
الديسك من الامتلاء)

(`to protect your disk from filling up with temporary files`).

○ `logrotate.timer` : تقوم بعمل روتيت وتدوير لملفات السجلات بانتظام)
(`to rotate log files`) للحفاظ على مساحة التخزين.

○ **خدمة الـ sysstat** : تعمل على جمع إحصائيات النظام)
(`collect sysstat / system statistics`) بشكل دوري كل خمس دقائق لمراقبة
أداء الموارد والـ performance.

1. تثبيت حزمة ال Sysstat:

- الأمر المستخدم:



Bash

```
dnf install sysstat
```

- الملاحظة الفنية: تثبيت حزمة `sysstat` المسؤولة عن توفير أدوات مراقبة الأداء وجمع إحصائيات النظام.

2. فحص وتشغيل خدمة ال Sysstat:

- الأوامر المستخدمة:



Bash

```
systemctl status sysstat  
systemctl start sysstat  
systemctl status sysstat
```

- الملاحظة الفنية:

- عند فحص ال `status` أول مرة، هنلاحظ أن الخدمة `inactive (dead)` .
- بعد تنفيذ أمر ال `start` وفحص ال `status` مرة أخرى، هنلاحظ أن حالتها بقيت `active (exited)` لأنها خدمة تنفيذية تقوم بمهمة مجدولة وتنتهي بنجاح.

3. استعراض التايمرات النشطة في النظام:

○ الأمر المستخدم:



Bash

```
systemctl list-units --type=timer
```

- الملاحظة الفنية: هنا نلاحظ ظهور التايمر الخاص بالخدمة وهو `sysstat-collect.timer` ووظيفته الأساسية هي إطلاق أداة جمع إحصائيات ونشاط النظام بانتظام (Run system activity accounting tool every 10 minutes).

4. عرض إعدادات التايمر وتفاصيل جدولته:

○ الأمر المستخدم:



Bash

```
systemctl cat sysstat-collect.timer
```

- الملاحظة الفنية:
- يظهر محتوى ملف التايمر الإعدادي، وأهم سطر فيه هو `OnCalendar=*:00/10`.
- هذا السطر يعني أن التايمر يقوم بعمل تريجر (Trigger) أو إطلاق للخدمة المسؤولة عن جمع إحصائيات النظام كل 10 دقائق بانتظام.



Bash

```
sar -r
```

○ الملاحظة الفنية:

- استخدام أمر `sar` الشهير (ويعتبر من أهم الأدوات التي تنزل مع حزمة الـ `sysstat`) لعرض تقارير وإحصائيات استخدام الذاكرة وأداء النظام.
- توضيح هام حول توقيت ظهور البيانات: لو قمنا بتشغيل أمر الـ `sar` فوراً بعد تفعيل الخدمة، قد لا تجد بيانات ظهرت أو تجدها فارغة؛ والسبب في ذلك أن التايمر لا يعمل لمرة واحدة فقط عند التفعيل وخلاص، بل هو نظام مستمر يطلق عملية تجميع البيانات كل 10 دقائق بانتظام (مثل الدقيقة 0، 10، 20 وهكذا). لذلك، يجب أن تمر أول 10 دقائق على الأقل ويقوم التايمر بتشغيل الخدمة لتسجيل أول عينة بيانات في السجلات لكي تظهر معك عند استخدام الـ `sar`.

6. تعديل إعدادات التايمر وأسباب النقل للمسار `/etc/`:

- الأوامر المستخدمة (وتم تقسيم سطر النسخ الطويل ليكون واضحاً):



Bash

```
cp /usr/lib/systemd/system/sysstat-collect.timer \
/etc/systemd/system/sysstat-collect.timer

vi /etc/systemd/system/sysstat-collect.timer

systemctl daemon-reload
```


○ الملاحظات الاحترافية لسيستم أدمن:

- لماذا تم عمل نسخ للمسار `/etc/` بالتحديد؟ لأن الملفات الموجودة في `/usr/lib/systemd/system/` هي ملفات النظام الأصلية التابعة للشركة المزودة (Vendor files)، وأي تحديث مستقبلي للحزمة (Package Update) سيقوم بالكتابة فوقها ومسح تعديلاتك (Overwrite). لذلك نقوم بنسخها إلى مسار `/etc/systemd/system/` لأن مدير النظام (systemd) يمنح الأولوية دائماً لملفات الـ `etc/` ، ولضمان بقاء تعديلاتك آمنة.
- أمر `systemctl daemon-reload` : ضروري جداً بعد أي تعديل لكي يصبح مدير النظام (systemd) على دراية ووعي تام (aware) بالتعديلات الجديدة التي أجريتها.

7. استعراض كافة ملفات وخدمات الـ Sysstat:

○ الأمر المستخدم:



Bash

```
ls /usr/lib/systemd/system/sysstat*
```

- الملاحظة الفنية: استعراض كافة التايمرات والخدمات المرتبطة مثل (`sysstat-summary.timer` , `sysstat-collect.timer` , `sysstat-collect.service`) والتي تعمل معاً بتكامل تام ليقوم التايمر بعمل تريجر للخدمة في الوقت المحدد لتجميع إحصائيات النظام.

Cleaning temporary files manually

- Use the following **systemctl** command to view the contents of the **systemd-tmpfiles-clean.timer** unit configuration file.
- [user@host ~]\$ **systemctl cat systemd-tmpfiles-clean.timer.**
- In the preceding configuration the parameter **OnBootSec=15min** indicates that the service unit called **systemd-tmpfiles-clean.service** gets triggered 15 minutes after the system has booted up. The parameter **OnUnitActiveSec=1d** indicates that any further trigger to the **systemd-tmpfiles-clean.service** happens 24 hours after the service unit was activated last.

- Based on your requirement, you can change the parameters in the **systemd-tmpfiles-clean.timer** timer unit configuration file. For example, the value **30min** for the parameter **OnUnitActiveSec** triggers the **systemd-tmpfiles-clean.service** service unit 30 minutes after the service unit was last activated. As a result, **systemd-tmpfiles-clean.service** gets triggered every 30 minutes after bringing the changes into effect. تطبيق
- After changing the timer unit configuration file, use the **systemctl daemon-reload** command to ensure that **systemd** is aware of the change. This command reloads the **systemd** manager configuration.

```
[root@host ~]# systemctl daemon-reload
```

Configuration file precedence

- Configuration files can exist in three places:
 - **/etc/tmpfiles.d/*.conf**
 - **/run/tmpfiles.d/*.conf**
 - **/usr/lib/tmpfiles.d/*.conf**
- The files in **/usr/lib/tmpfiles.d/** are provided by the relevant RPM packages, and you should not edit these files.
- The files under **/run/tmpfiles.d/** are themselves volatile files, normally used by daemons to manage their own runtime temporary files.
- The files under **/etc/ tmpfiles.d/** are meant for administrators to configure custom temporary locations, and to override vendor-provided defaults.

مسارات ملفات الإعدادات الثلاثة وأسبقيتها (Configuration File Precedence) - مرتبة حسب الأولوية):

توجد ملفات الإعدادات الخاصة بالملفات المؤقتة في ثلاثة أماكن رئيسية داخل النظام، وهي مرتبة تصاعدياً من حيث الأولوية والقوة:

1. مسار الأدمن (`etc/tmpfiles.d/*.conf/`) - [الأولوية الأعلى والقصوى]:
 - هذا المسار مخصص لك أنت كـ **Administrator** لتكوين وتخصيص إعدادات مؤقتة جديدة، أو لعمل **Override** (تجاوز واستبدال) لأي إعدادات افتراضية أخرى.
2. مسار التشغيل المؤقت (`run/tmpfiles.d/*.conf/`) - [الأولوية المتوسطة]:
 - ملفات متطيرة أو مؤقتة وقت التشغيل (`volatile files`)، وعادة ما تستخدمها الخدمات والديمونز لإدارة ملفات المؤقتة الخاصة أثناء عمل النظام.
3. مسار الشركة المزودة (`usr/lib/tmpfiles.d/*.conf/`) - [الأولوية الأقل أو الأساسية]:
 - هذه هي الملفات الأصلية التي تأتي جاهزة مع حزم الـ **RPM** (`provided by RPM packages`)، ويُحذر تماماً من تعديلها مباشرة.

قاعدة الأسبقية والـ Override (كيف يعمل الترتيب؟):

- ما علاقة الترتيب بالأولوية؟
 - نظراً لأن هذه المسارات مرتبة بالأولوية، فإن النظام إذا وجد ملفين نفس الاسم في مسارين مختلفين، فهو يمنح الأولوية المطلقة للملف الموجود في المسار الأعلى (`/etc/`) ثم `/run/` وأخيراً `/usr/lib/`).
 - **تطبيق عملي:** لو قمت بإنشاء ملف إعدادات في `/etc/tmpfiles.d/` وأعطيته نفس اسم ملف موجود في `/usr/lib/tmpfiles.d/`، فإن النظام سيتجاهل ملف الشركة المزودة تماماً ويطبق ملفك أنت (Override)، وهي الطريقة الآمنة لتعديل الإعدادات دون الخوف من ضياعها عند تحديث الحزم!

مثال عملي على ملف الإعدادات وقاعدة الـ Override:

- الوصول للمسار الأصلي:
لو رحنا للمسار الأصلي اللي هو `usr/lib/tmpfiles.d/tmp.conf/` وفتحت الملف هتلاقيه مديك سطر كذا:



Python

```
d /tmp 1777 root root 10d
```

ومعنى السطر ده باختصار: إنه بيتأكد من وجود الدايركتوري، ولو موجود يمسح الملفات اللي فيه لو عدى عليها 10 أيام بدون استخدام.

- إمتى نعمل Override؟
لو شايف إن الـ 10 أيام كتيرة ومحتاج مثلاً تخلوها يومين اثنين فقط.

○ الخطأ والصح:

- الخطأ: انك تروح للملف الأصلي ده وتعديل فيه مباشرة.
- الصح: لا، انت هتأخذ كوبي من الملف وتحطه في مسار الـ `etc/tmpfiles.d/tmp.conf/`، وتفتحه وتعديل فيه وتكتب القيمة الجديدة، وبعدين تحفظ الملف، وبعد كده تعمل أمر `systemctl daemon-reload` بس كده.

○ النتيجة النهائية:

النظام لما يجي يشغل الخدمة هيقرا الملف اللي في الـ `/etc/` فيعتبر هو القانون الأساسي ويتجاهل الملف الأصلي تماماً، بس كده.

التذكير الأول: الفرق بين الـ 15 دقيقة والـ 10 أيام (الأزمة في النظام):

- الـ 15 دقيقة (15min):

- تتبع إيه؟ خاصة بالتايمر أو المنبه (timer).

- معناها إيه؟ بتحدد وقت البداية بعد الإقلاع؛ يعني بتقول للنظام: "استنى 15 دقيقة بعد ما السيرفر يفتح، وبعدين شغل خدمة التنظيف لأول مرة".

- الـ 10 أيام (10d):

- تتبع إيه؟ خاصة بملف إعدادات الملفات المؤقتة (tmp.conf).

- معناها إيه؟ بتحدد عمر الملفات (Age)؛ يعني بتقول للأمر: "أي ملف جوه مجلد الـ tmp/ عدى عليه 10 أيام بدون استخدام، احذفه فوراً".

التذكير الثاني: رحلة التنظيف (مين بيعمل إيه بالضبط؟):

- المنبه (timer):

- ده عبارة عن "ساعة رنين" أو منبه هدفه الوحيد إنه يعرف إن الوقت جه (سواء بعد 15 دقيقة، أو كل يوم). هو مش بيمسح أي حاجة بيده خالص!

- الخدمة والأمر التنفيذي (service. & systemd-tmpfiles --clean):

- أول ما المنبه يرن، بيروح يصحي الخدمة المسؤولة، والخدمة بتشغل الأمر، والأمر ده هو اللي بينفذ عملية الحذف الفعلي للملفات القديمة اللي عدت العمر المسموح (الـ 10 أيام، أو الـ 2 يوم لو عملت Override).

تتابع الأوامر والخطوات بالتفصيل:

- أمر نسخ ملف التايمر إلى مسار الـ `/etc/` (مقسم على سطرين):



Bash

```
cp /usr/lib/systemd/system/systemd-tmpfiles-clean.timer \  
/etc/systemd/system/systemd-tmpfiles-clean.timer
```

- شرح اللي عملناه: أخذنا نسخة من ملف التايمر الأصلي الخاص بالشركة المزودة ونقلناها إلى مسار الـ `/etc/` عشان نعمل تجاوز آمن (`Override`) ونضمن إن تعديلاتنا مش هتطير لو الحزمة اتحدّثت في المستقبل.

- أمر فتح ملف التايمر للتعديل:



Bash

```
vi /etc/systemd/system/systemd-tmpfiles-clean.timer
```

- شرح اللي عملناه: فتحنا النسخة اللي في الـ `/etc/` باستخدام محرر النصوص `vi` عشان نعدل فترات التوقيت.

- `systemctl daemon-reload`

- `systemctl cat systemd-tmpfiles-clean.timer`

- شرح اللي عملناه: عرضنا محتوى ملف التايمر لكي نتأكد أن التعديلات الجديدة ظهرت وبقت بنجاح في النظام.

`/cd /usr/lib/tmpfiles.d`

○

○ شرح اللي عملناه: انتقلنا للمجلد الأساسي الذي يحتوي على كل ملفات الإعدادات الخاصة بإدارة الملفات المؤقتة.

`ls`

○

○ شرح اللي عملناه: استعرضنا محتويات المجلد لنرى أن كل خدمة أو مكون في النظام له ملف إعدادات خاص بيه مستقل لإدارة ملفاته المؤقتة.

`cat tmp.conf`

○

○ شرح اللي عملناه: فتحنا ملف الإعدادات الخاص بالـ `tmp/` لعرض محتواه وقواعد التنظيف الداخلية.

○ السطور الداخلية لملف الإعدادات:



Bash

```
d /tmp 1777 root root 10d
d /var/tmp 1777 root root 30d
```

○ شرح اللي عملناه: هذه السطور تأمر النظام بالتأكد من وجود المجلد (أو إنشائه)، وتحديد الصلاحيات (1777) والمالك (root root)، وحذف أي ملف جواه عدى عليه 10 أيام بدون استخدام بالنسبة لـ `tmp/`، و30 يوماً بالنسبة لـ `var/tmp/`.

تتابع الأوامر والخطوات:

- إضافة السطر الجديد في ملف الإعدادات باستخدام الـ `vi` :



Plaintext

```
d /tmp1 0700 root root 30s
```

- شرح اللي عملناه: فتحنا ملف `tmp.conf` بالمحرر وضيفنا السطر ده عشان تنشئ مجلد جديد اسمه `tmp1/` بصلاحيات `0700` ومالك `root:root` ، وخليتنا عمر الملفات `30` ثانية (`30s`) فقط عشان نقدر نختبر ونشوف عملية الحذف بعيننا بسرعة من غير ما نقعد نستحمل ساعات أو أيام.

```
systemd-tmpfiles --create tmp.conf
```

-

- شرح اللي عملناه: شغلنا أمر الإنشاء يدوياً عشان النظام يفعل القاعدة فوراً وينشئ مجلد `tmp1/` بالصلاحيات الجديدة، لأننا مش عايزين نستنى معاد الـ Timer الأصلي.

```
cd /tmp1
```

-

- شرح اللي عملناه: انتقلنا ودخلنا جوه المجلد الجديد `tmp1/` اللي لسه متعرف ومتكريت حالياً.

```
touch file1 file2
```

-

- شرح اللي عملناه: أنشأنا ملفين وهميين (`file1` و `file2`) جوه المجلد عشان نعمل عليهم التجربة ونختبر بيهم الحذف.

sleep 30 ○

- شرح اللي عملناه: عملنا إيقاف مؤقت لمدة 30 ثانية عشان نتأكد إن العمر المخصص للملفات (30s) اللي حددناه في الإعدادات قد انقضى.

ls ○

- شرح اللي عملناه: استعرضنا محتويات المجلد للتأكد من وجود الملفات (file1 و file2) قبل عملية التنظيف.

systemd-tmpfiles --clean /etc/tmpfiles.d/tmp.conf ○

- شرح اللي عملناه: نفذنا أمر التنظيف يدوياً باستخدام ملف الإعدادات المعدل، وعشان الـ 30 ثانية خلصت، النظام دخل جوه الفايل ونفذ الحذف الفوري للملفات القديمة!

إمتى clean-- و create-- بيتنفذوا في النظام؟

- أمر الإنشاء (create--):

- بيتنفذ تلقائياً أول ما الجهاز يبدأ إقلاع (Boot) من خلال خدمة مخصصة، ووظيفته إنه يبص على ملفات الإعدادات وينشئ أي مجلد مش موجود (زي مجلد tmp1/ اللي عملناه) ويدي له الصلاحيات والمالك الصح.

- أمر التنظيف (clean--):

- بيتنفذ تلقائياً بشكل دوري ومنتظم عن طريق المنبه والخدمة المرتبطين ببعض (timer. و service.). أول ما الجهاز بيعمل إقلاع والوقت المحدد للتايمر بيخلص (مثلاً بعد 15 دقيقة أو 30 دقيقة)، المنبه بيضرب ويصحى الخدمة، فتروح تبص في الملفات، ولو لقيت ملفات عدى عليها العمر المحدد (زي الـ 30 ثانية أو الـ 10 أيام)، بتحذفها فوراً.

Summary

In this chapter, you learned:

- Jobs that are scheduled to run once in the future are called deferred jobs or tasks.
- Recurring user jobs execute the user's tasks on a repeating schedule.
- Recurring system jobs accomplish administrative tasks on a repeating schedule that have system-wide impact.
- The **systemd** timer units can execute both the deferred or recurring jobs.



خطوات تثبيت ال Mount بشكل دائم باستخدام `etc/fstab/`

○ **الخطوة الأولى:** استخدم أمر `blkid` لمعرفة المعرف الفريد (UUID) للبارتشن أو الفلاشة المراد تثبيتها.

○ **الخطوة الثانية:** أنشئ مجلد جديد على النظام لتستقبل فيه الملفات (Mount Point) باستخدام أمر `mkdir /mnt/myfolder`.

○ **الخطوة الثالثة:** افتح ملف إعدادات التثبيت الدائم بصلاحيات الروت باستخدام أمر `sudo nano /etc/fstab`.

○ **الخطوة الرابعة:** أضف سطر جديد في نهاية الملف بالصيغة الصحيحة، مع كتابة نوع نظام الملفات المناسب مثل `ext4` أو `xfs` (حسب نوع الفلاشة أو البارتشن عندك)، وليكن هكذا:

```
UUID=xxx... /mnt/myfolder xfs defaults 0 0
```

○ **الخطوة الخامسة:** احفظ الملف ثم نفذ أمر `sudo mount -a` للاختبار؛ **ملحوظة هامة:** لو الأمر لم يُخرج أي رسائل أخطاء وعاد سطر التيرمينال طبيعياً، فهذا معناه أن إعداداتك صحيحة 100%، أما لو ظهر أي خطأ فهناك مشكلة في الكتابة يجب مراجعتها قبل عمل Reboot.